



第七章 回溯法





目录

- 7.1 回溯一般方法
- 7.2 回溯法的效率估计
- 7.3 n -皇后问题
- 7.4 子集和问题
- 7.5 图着色问题
- 7.6 小结





7.1 回溯的一般方法

- 方法适用的问题特点
- 多米诺性质
- 解空间树
- 问题建模
- 动态树中的结点类型
- 动态树结点生成方式
- 回溯法的设计思想
- 算法7.1 回溯法的非递归算法
- 算法7.2 回溯法的递归算法



回溯：多米诺性质
动规：最优性原理

计算机科学与
技术学院



方法适用的问题特点

- 方法适用于解决多阶段决策问题，也称为组合问题
 - 问题的解向量用元组来表示，元素 x_i 通常取自于某个有穷集 S_i ， $1 \leq i \leq n$ ， n 表示问题规模
 - 固定长 n -元组 $(x_1, \dots, x_n)$
 - 不定长 k -元组 $(x_1, \dots, x_k)$ ， $k \leq n$
 - 问题满足多米诺性质✖
 - 问题寻找满足约束条件的一个解或所有解；有时则要寻找最优解，此时问题中还存在目标函数

组合搜索

组合优化



细分多阶段决策问题

- 多阶段决策问题/组合问题
 - 组合搜索：关注解的存在性
 - 问题描述中仅存在约束条件
 - 问题希望找到一个可行解/所有可行解
 - 组合优化：关注解的最优性
 - 问题描述中还存在目标函数
 - 问题希望找到一个具有极小值/极大值的可行解，即最优解
 - 也称为极小化/极大化问题

所有可能解

可能解中最优解



多米诺性质

- 多米诺性质

- 设 $P(x_1, \dots, x_i)$ 是关于向量 $(x_1, \dots, x_i)$ 的某种性质的判定，当 $P(x_1, \dots, x_{i+1})$ 为真时，一定有 $P(x_1, \dots, x_i)$ 为真， $0 < i < n$

在 x_i 处不成立了，以后所有的都不可能成立

- 根据多米诺性质，如果 $P(x_1, \dots, x_i)$ 不成立，则 $P(x_1, \dots, x_{i+1})$ 亦不成立

- 一个满足多米诺性质的组合问题

- 是指能够根据约束条件和目标函数不断检验正在构造的部分解向量 $(x_1, \dots, x_i)$ ， $0 < i < n$ ，一旦发现不成立，则无需考虑后续 $x_{i+1}, \dots, x_n$ 的取值

思考：利用多米诺性质能够提高算法效率的关键，它会如何起作用？



问题的多米诺性质是回溯法提高算法效率的关键

- 设约束函数 B 实现问题的约束条件, 对于 n -元组 $(x_1, \dots, x_n)$, $x_i \in S_i$,
- 硬性处理
 - $|S_i|=m_i$, 向量个数 $m=m_1 \times m_2 \times \dots \times m_n$, 对这 m 个 n -元组逐一检测是否满足 $B(x_1, \dots, x_n)$, 从而找出问题的解。
- 回溯法利用多米诺性质
 - 约束函数 B 可以提前检验正在构造中的部分向量 $(x_1, \dots, x_i)$, 如果发现不能导致问题的解, 终止该向量继续构造, 即不再构造 $x_{i+1}, \dots, x_n$ 的取值, 从而减少了 $m_{i+1} \times \dots \times m_n$ 个向量。



解空间树



回溯法解决问题的过程就是在解空间树上搜索答案状态结点的过程

取值范围

- 显式约束：每个 x_i 的取值集合 S_i ，可能与问题实例有关，也可能无关
- 隐式约束：描述了 x_i 彼此相关的情况，与问题实例有关
- 解空间：满足显式约束条件的所有元组
- 解空间树：基于解空间画成的树形状

对应约束函数

- 问题状态：解空间树中的所有结点。

满足显式约束条件

- 解状态 X ：由根到结点 X 的路径确定了解空间中的一个元组

- 答案状态 X ：由根到解状态结点 X 的路径确定了问题的一个解。

还满足隐式约束条件

定义
(填充
选择)





问题建模

- 确定元组表达形式
 - 固定长元组：用大小固定的 n -元组 $(x_1, \dots, x_n)$ 表示
 - 不定长元组：用大小可变的 k -元组 $(x_1, \dots, x_k)$ 表示, $k \leq n$
- 确定约束条件：显示约束和隐式约束
- 检验问题满足多米诺性质
- 确定解空间树：问题状态、解状态和答案状态





以子集和问题为例

- 问题描述：
 - 已知 $n+1$ 个正数： $w_i (1 \leq i \leq n)$ 和 M ，要求找出 w_i 的和是 M 的所有子集。
- 问题实例：
 - $n=4$, $(w_1, w_2, w_3, w_4) = (11, 13, 24, 7)$, $M=31$ 。
 - 可行解1： 11, 13, 7
 - 可行解2： 24, 7





固定长元组表达

- n -元组 $(x_1, \dots, x_n)$
- 显式约束: $x_i \in \{0, 1\}, 1 \leq i \leq n$; 如果选择 w_i , 则 $x_i = 1$; 否则 $x_i = 0$
- 隐式约束: $\sum w_i x_i = M, 1 \leq i \leq n$
- 多米诺性质: 如果部分解向量大于 M , 则包含它的解向量也大于 M
- 解空间共计 2^n 个元组

$n=4, (w_1, w_2, w_3, w_4) = (11, 13, 24, 7), M=31$ 。 解空间: $2^4=16$ 个元组

可行解1: 11, 13, 7

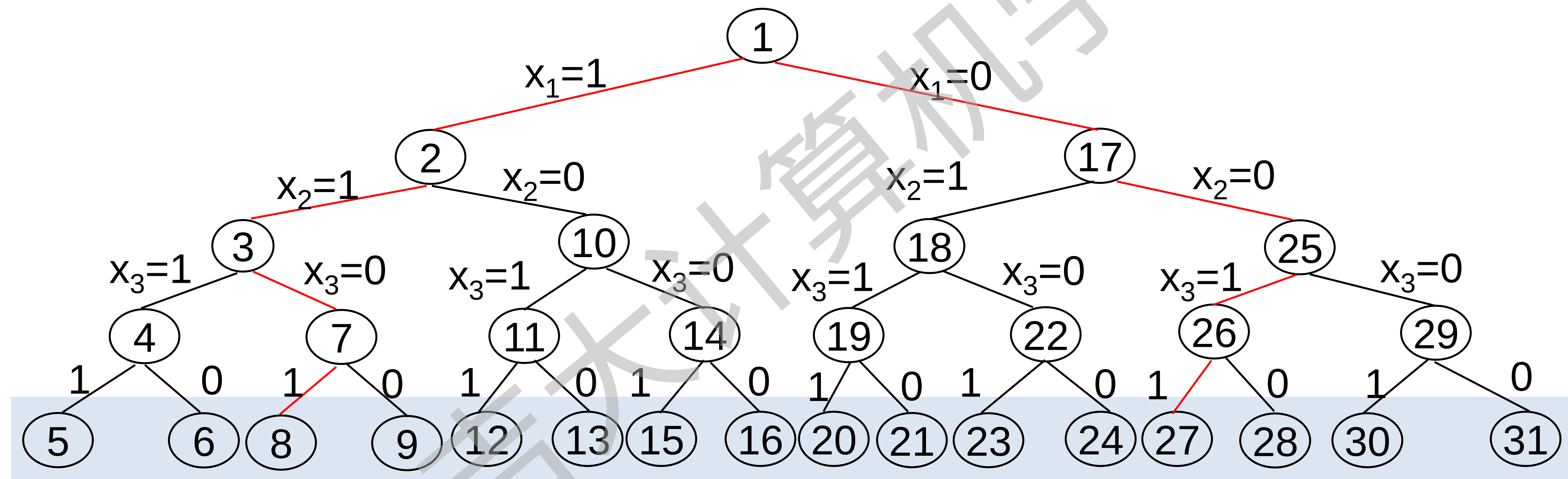
可行解1: (1, 1, 0, 1)

可行解2: 24, 7

可行解2: (0, 0, 1, 1)



- 子集和问题的4-元组表达的解空间树
 - 问题状态：全部结点31个 16 + 15
 - 解状态：叶结点16个 叶 内
 - 答案状态：当前实例2个



从根结点到叶结点的一条路径确定解空间中的一个元组

不定长元组表达

避免重复情况,
如(1,2,4)和(1,4,2)

- k -元组 $(x_1, \dots, x_k)$, $k \leq n$
- 显式约束: $x_i \in \{j \mid j \text{ 是 } w_j \text{ 的下标}, 1 \leq j \leq n\}$, $1 \leq i \leq k$ 。 $x_i < x_{i+1}$, $1 \leq i < k$
- 隐式约束: 相应的 w_i 的和等于 M
- 多米诺性质: 如果部分解向量大于 M , 则包含它的解向量也大于 M
- 解空间共计 2^n 个元组

$n=4$, $(w_1, w_2, w_3, w_4) = (11, 13, 24, 7)$, $M=31$ 。 解空间: $2^4=16$ 个元组

可行解1: 11, 13, 7

可行解1: (1, 2, 4)

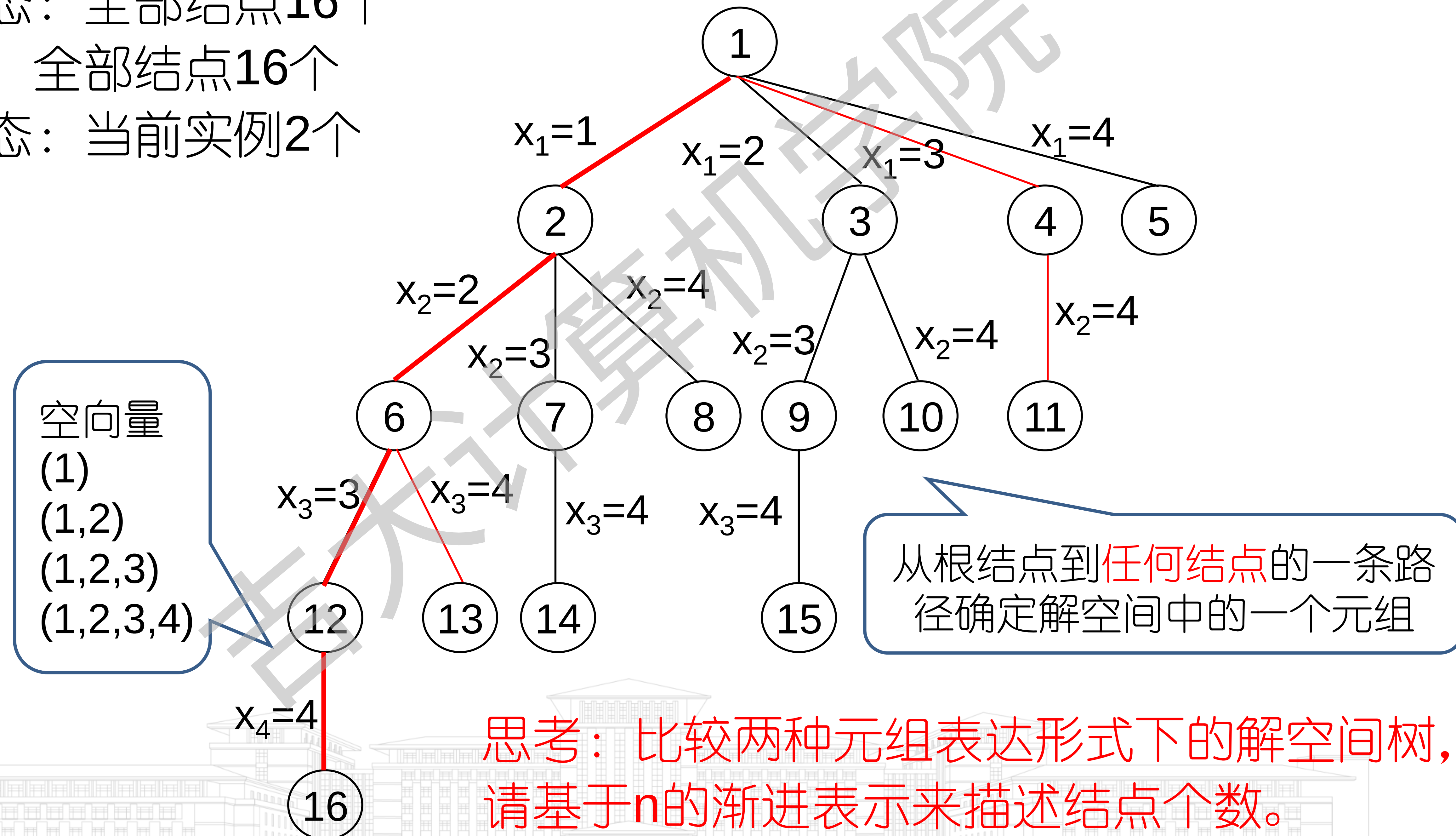
可行解2: 24, 7

可行解2: (3, 4)



子集和问题的 $n=4$ 时， k -元组表达的解空间树

- 问题状态：全部结点16个
- 解状态：全部结点16个
- 答案状态：当前实例2个



动态树中的结点类型

- 静态树: 即解空间树, 树结构与所要解决的问题实例无关。
- 动态树: 树结构与实例相关, 在求解过程中生成结点。
 - 活结点: 自己已经生成而其儿子结点还没有全部生成的结点。
 - E-结点(正在扩展的结点): 当前正在生成其儿子结点的活结点。
 - 死结点: 不再进一步扩展或者其儿子结点已全部生成的结点。



动态树结点生成方式

- 第一种方式

- 当前的E-结点R一旦生成一个新的儿子结点C, 这个C结点就变成一个新的E-结点, 当检测完了子树C后, R结点就再次成为E-结点, 生成下一个儿子结点。

该方法基于深度优先, 称为回溯法。

- 第二种方式:

- 当前结点一旦成为E-结点, 就一直处理到变成死结点为止。其生成的儿子结点加入到活结点表中, 然后再从活结点表中选择下一个新的E-结点。

该方法称为分支限界法。

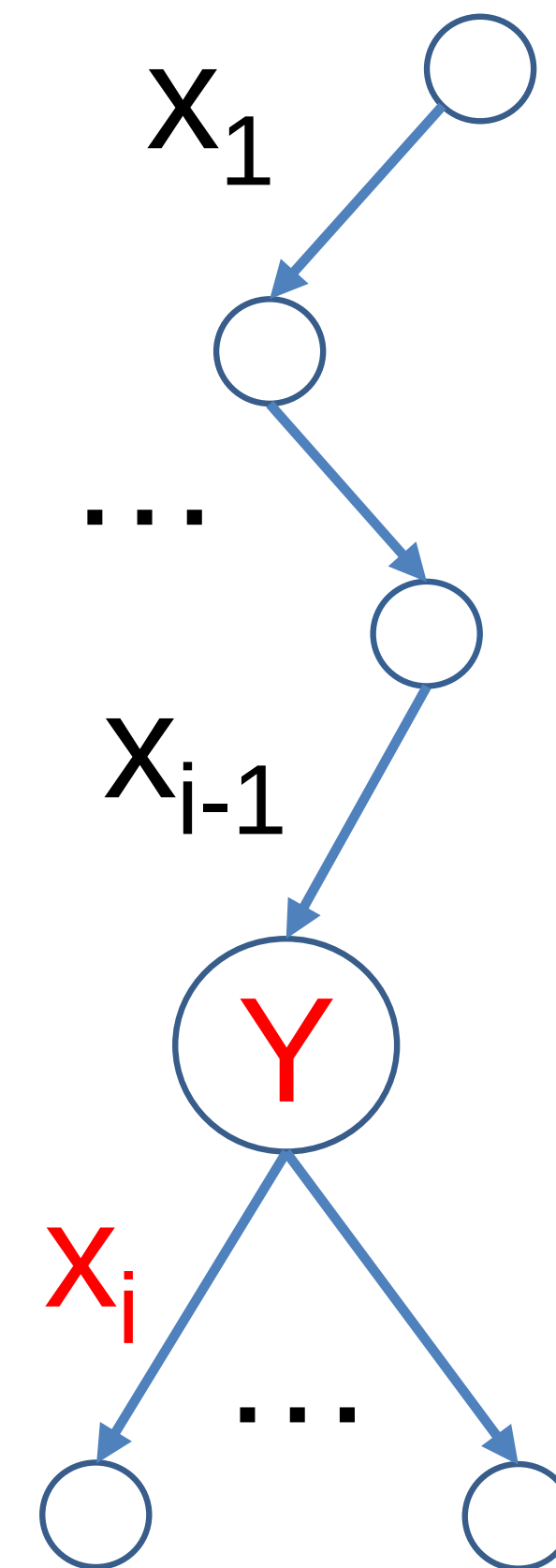


回溯法的设计思想

- 针对问题定义解空间树结构：元组、显式约束条件、**隐式约束条件**。
- 检验问题满足多米诺性质。
- 以深度优先方式搜索解空间树，在搜索过程中使用**约束函数B**避免无效搜索。
 - 对当前**E-结点R**，一旦生成一个新的儿子结点**C**，**C**就成为新的**E-结点**，搜索向纵深方向移动。
 - 如果当前**E-结点**不能满足约束函数**B**，或者再没有儿子节点，则其成为死结点，搜索不再向纵深方向移动，转而向上回溯至最近的一个活结点，该活结点成为新的**E-结点**。
 - 搜索过程直至找到所要求的解或解空间树中已没有活结点时为止。

回溯法的形式化描述

- 假设要找出所有的答案结点
- $(x_1, x_2, \dots, x_{i-1})$ 是状态空间树中由根出发的一条路径，到达结点 Y
- $T(x_1, \dots, x_{i-1})$ 是元素 x_i 的集合，对于每一个 x_i ， $(x_1, x_2, \dots, x_{i-1}, x_i)$ 是一条由根到结点 Y 的一个儿子结点的路径
- 对于约束函数 B_i ，如果路径 $(x_1, x_2, \dots, x_{i-1}, x_i)$ 不可能延伸到一个答案结点，则 $B_i(x_1, x_2, \dots, x_i)$ 取假值，否则取真值



算法7.1 回溯法的非递归算法



```
procedure BACKTRACK(n)
  int k, n
  local X(1: n)
  k ← 1
  while (k>0) do
    if (还剩有没检验的X(k)使得 $X(k) \in T(X(1) \dots X(k-1))$ 
      and  $B(X(1) \dots X(k)) = \text{TRUE}$ )
    then if (X(1) ... X(k))是一条抵达答案结点的路径)
      then print ( X(1) ... X(k))
      endif
      k ← k+1
    else k ← k-1
    endif
  repeat
end BACKTRACK
```



```
// 假设 n 是全局变量或者作为参数传递给函数
extern int n;
bool B(int *X, int k); // 约束函数B的声明，需要自行定义
void printSolution(int *X, int k); // 打印解的函数，需要自行定义

void BACKTRACK(int n) {
    int X[100]; // 假设n不会超过100，或者根据实际情况调整数组大小
    int k = 1;
    while (k > 0) {
        // 查找下一个满足约束条件的X[k]
        bool found = false;
        while (!found && X[k] < n - 1) { // 这里假设Si是从0到n-1的整数集
            X[k]++;
            if (B(X, k)) { // 使用约束函数检查当前选择是否可行
                found = true;
            }
        }

        if (found) {
            if (k == n) { // 如果找到了一个完整的解
                printSolution(X, k);
            } else {
                k++; // 移动到下一个决策点
                X[k] = -1; // 初始化下一层的选择为非法值，以便于在查找时增加它
            }
        } else {
            k--; // 回溯
        }
    }
}
```





算法7.2 回溯法的递归算法

```

procedure RBACKTRACK(k)
global X(1:n);
int k, n;
for ( 满足下式的每个X(k),  $X(k) \in T(X(1)...X(k-1))$ 
    and  $B(X(1),...X(k))=true$ ) do
    if  $(X(1),...,X(k))$ 是一条抵达答案结点的路径 then
        print  $(X(1)...X(k))$  endif
    call RBACKTRACK(k+1)
repeat
end RBACKTRACK
    
```

进入算法时, 解向量X中的前k-1个分量 $X(1) \dots X(k-1)$ 已经被赋值




```
// 假设 n 是全局变量或者作为参数传递给函数
extern int n;
bool B(int *X, int k); // 约束函数B的声明，需要自行定义
void printSolution(int *X, int k); // 打印解的函数，需要自行定义

void RBACKTRACK(int k, int n, int *X) {
    for (int Xk = 0; Xk < n; Xk++) { // 这里假设Si是从0到n-1的整数集
        X[k] = Xk;
        if (B(X, k)) { // 使用约束函数检查当前选择是否可行
            if (k == n - 1) { // 如果找到了一个完整的解
                printSolution(X, k);
            } else {
                RBACKTRACK(k + 1, n, X); // 递归调用以继续下一个决策点
            }
        }
    }
}
```

```
// 调用RBACKTRACK时，需要初始化解向量X中的前k-1个分量，并设置初始的k值
// 示例调用：
// int X[100]; // 假设n不会超过100，或者根据实际情况调整数组大小
// RBACKTRACK(0, n, X);
```




7.2 回溯法的效率分析

- 决定回溯法效率的因素
- 回溯法的效率估计
- 蒙特卡罗方法的一般思想
- 算法7.3 效率估计算法
- 蒙特卡罗方法的特点



决定回溯法效率的因素

- 生成下一个 $X(k)$ 的时间
 - 生成一个结点的时间
- 满足显式约束条件的 $X(k)$ 的数目
 - 子结点的数量
- 约束函数 B_i 的计算时间
 - 检验结点的时间
- 对于所有的 i , 满足 B_i 的 $X(k)$ 的数目
 - 通过检验的结点数量

B_i 能够大大减少生成的结点数, 但在计算时间和减少程度上要进行折中

约束函数 B_i 剪枝作用: 避免生成那些不包含可行解的子树。

思考: 哪一个因素体现出不同实例产生的结点数不同?

第四个

回溯法的效率估计

- 如果解空间的结点数是 2^n 或 $n!$ ，易知，回溯算法最坏情况下的时间复杂度为 $O(p(n)2^n)$ 或 $O(q(n)n!)$ ，其中 $p(n)$ 和 $q(n)$ 为 n 的多项式

B_i 花费的时间等

- 由于回溯法对同一问题不同实例的巨大差异，在 n 很大时，对某些实例是十分有效的。因此，在采用回溯法计算某个实例之前，应估算其工作能效
- 用回溯算法处理一棵树所要生成的结点数，可以用蒙特卡罗方法估算出来

估计活结点的个数，
即动态树结点个数

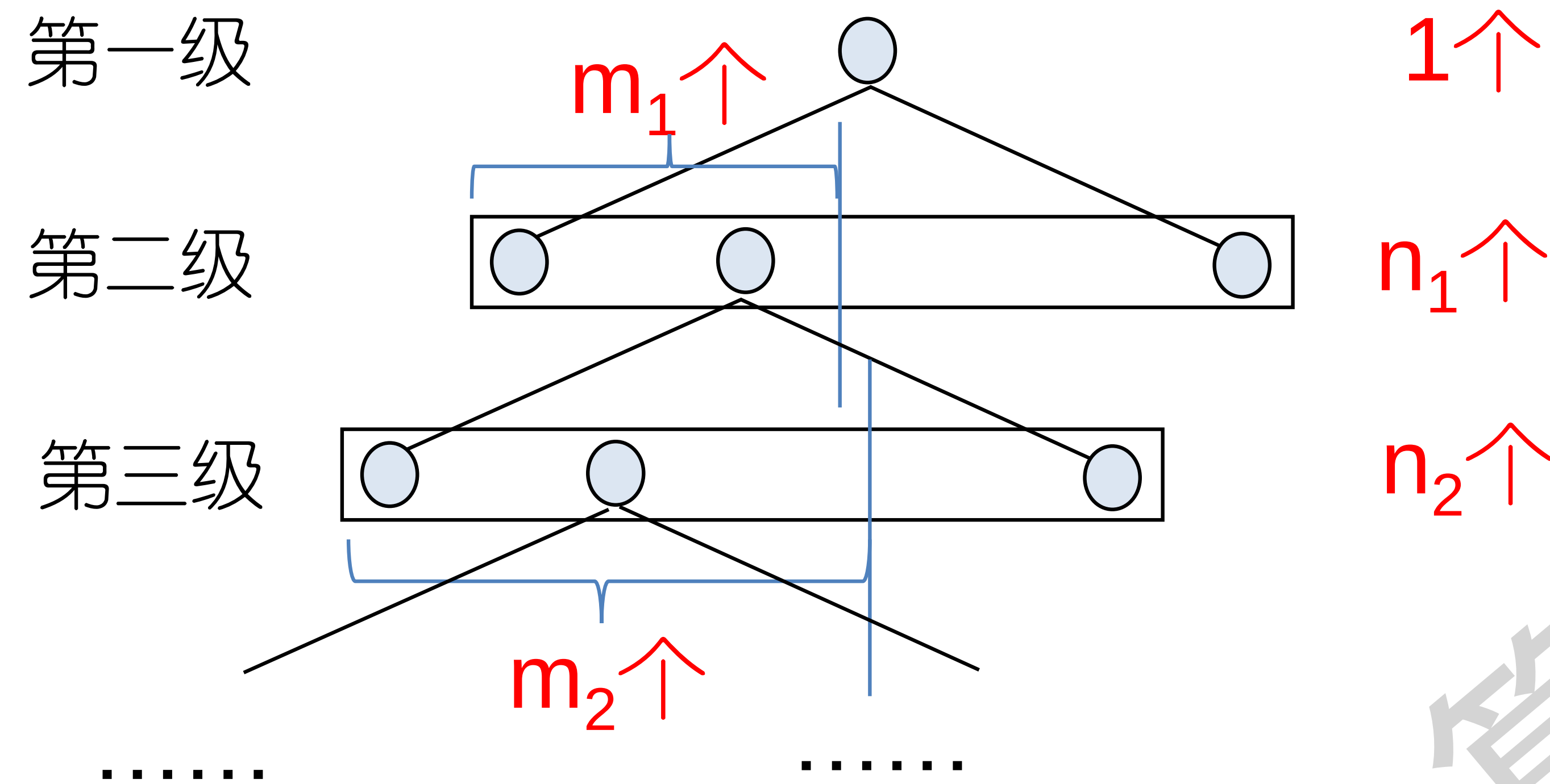


蒙特卡罗方法的一般思想

- 假定约束函数是**固定**的
 - 在状态空间中生成一条随机路径。
 - 设 x 是这条路径上的位于第 i 级的一个结点。
 - 设约束函数确定 x 的可用儿子结点的数目为 m_i 。
 - 从这 m_i 个儿子结点中随机选中一个，重复上述过程，直到当前结点是叶结点或者儿子结点都被限界为止。

不受限界的结点的估计数: $m = 1 + m_1 + m_1 * m_2 + m_1 * m_2 * m_3 + \dots$
 m_i 表示第 i 级结点平均没受限界的儿子结点数。





第一级通过**B**函数检验的结点个数有多少个？

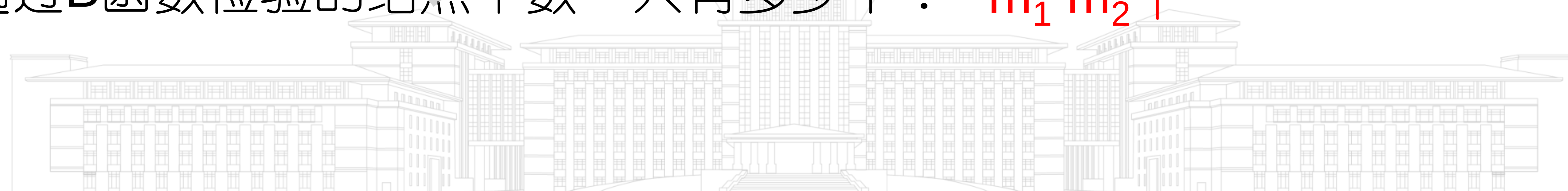
$1 \uparrow$

第二级通过**B**函数检验的结点个数有多少个？

$m_1 \uparrow$

从第二级中随机选中一个结点，它的子结点共有 m_2 个满足**B**函数检验，推断第三级通过**B**函数检验的结点个数一共有多少个？

$m_1 m_2 \uparrow$



算法7.3 效率估计算法



Procedure ESTIMATE() //程序沿着状态空间树中一条随机路径产生
这棵树中不受限界结点的估计数//

$m \leftarrow 1$; $r \leftarrow 1$; $k \leftarrow 1$

loop

$T_k \leftarrow \{X(k) : X(k) \in T(X(1), \dots, X(k-1)) \text{ and } B_k(X(1), \dots, X(k))\}$

if SIZE(T_k)=0 then exit endif

$r \leftarrow r * \text{SIZE}(T_k)$ //第 k 级的结点总数

$m \leftarrow m + r$ //前 k 级的结点总数

$X(k) \leftarrow \text{CHOOSE}(T_k)$ //从 T_k 中随机地挑选一个元素

$k \leftarrow k + 1$

repeat

return m

end ESTIMATE



蒙特卡罗方法的特点

- 优点
 - 适用于找到所有答案结点的情况
 - 约束函数固定不变，计算方便，对树中同一级结点均适用
- 缺点
 - 只求一个解时，生成的结点数远小于 m
 - 随着检索的进行，约束函数应该更强，使得 m 的值更小





7.3 n-皇后问题

- 问题描述
- 解空间树
- 问题分析
- 算法7.4 约束函数
- 算法7.5 n-皇后问题的回溯算法
- 效率估计



问题描述

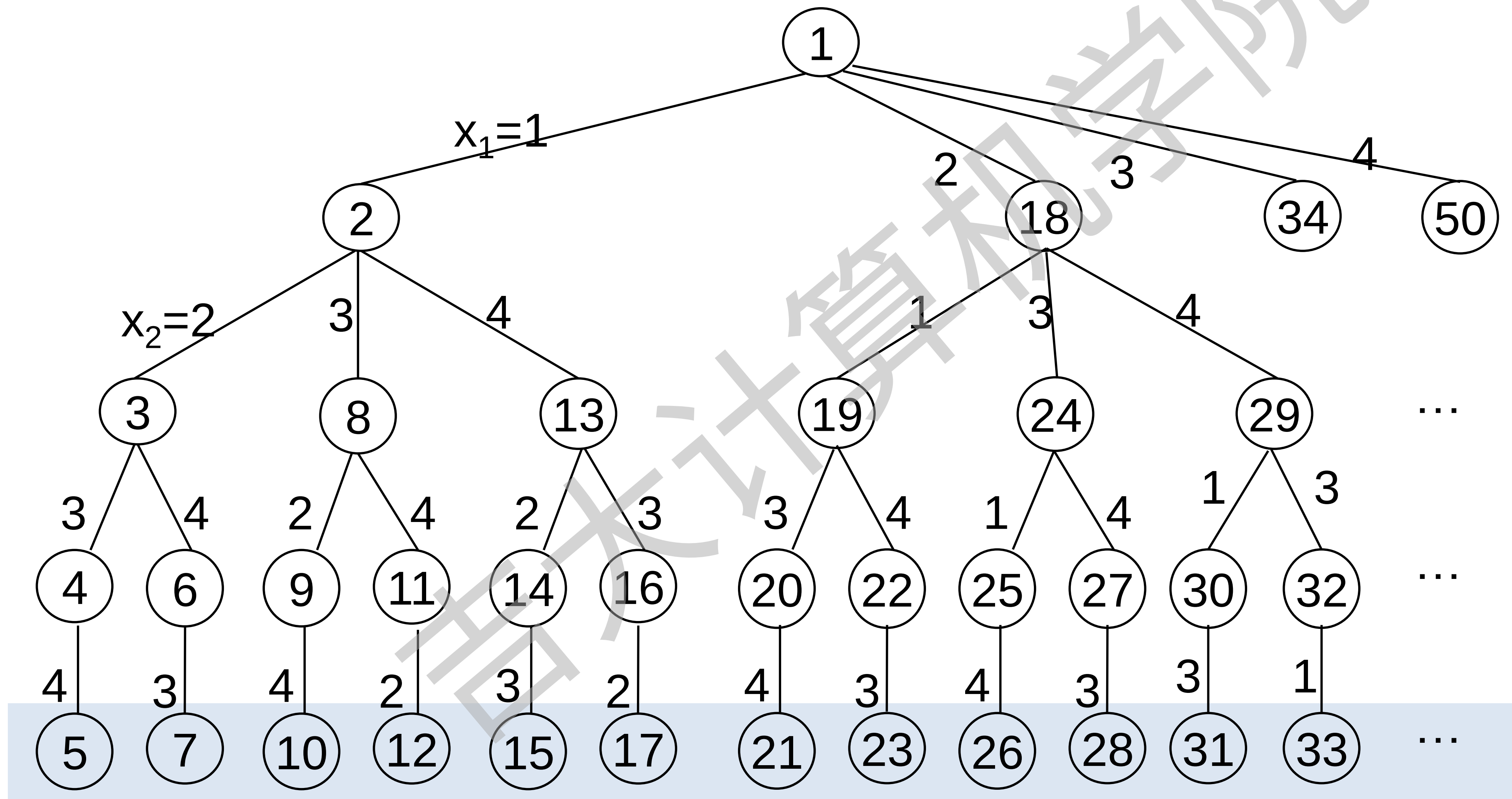
- n-皇后问题：
 - 在一个 $n \times n$ 棋盘上放 n 个皇后，使每两个皇后之间都不能互相“攻击”，即使得每两个皇后都不能在**同一行、同一列及同一条斜角线上**。
- 基于回溯法求解：
 - n-元组 $(x_1, \dots, x_n)$ ：表示皇后 i 放在 i 行 x_i 列上。 解空间： $n!$
 - 显式约束条件： $x_i \in \{1, 2, \dots, n\}$, $1 \leq i \leq n$ ，且没有两个 x_i 可以相同
 - 隐式约束条件：没有两个皇后可以在同一条斜角线上。



解空间树

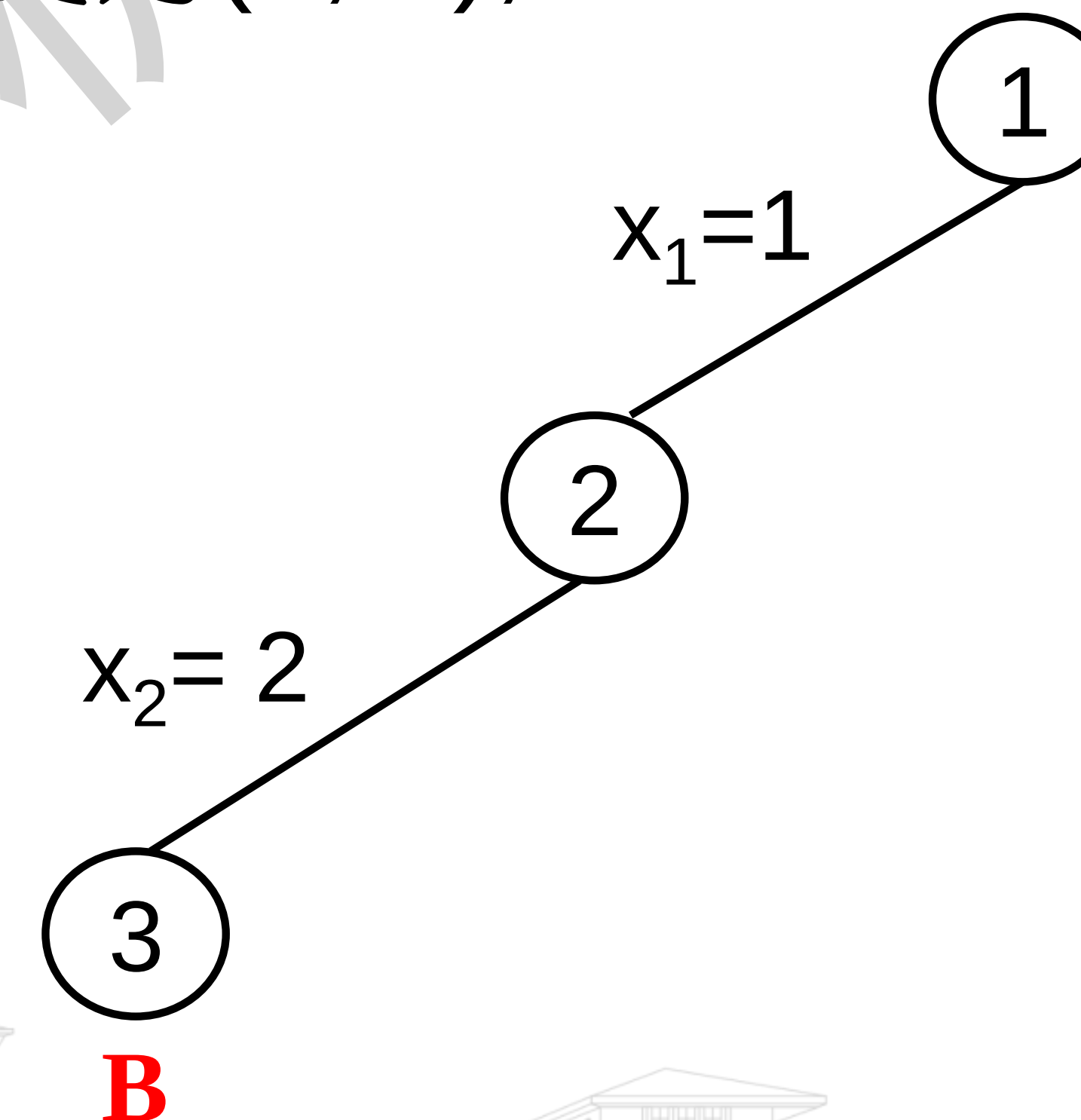
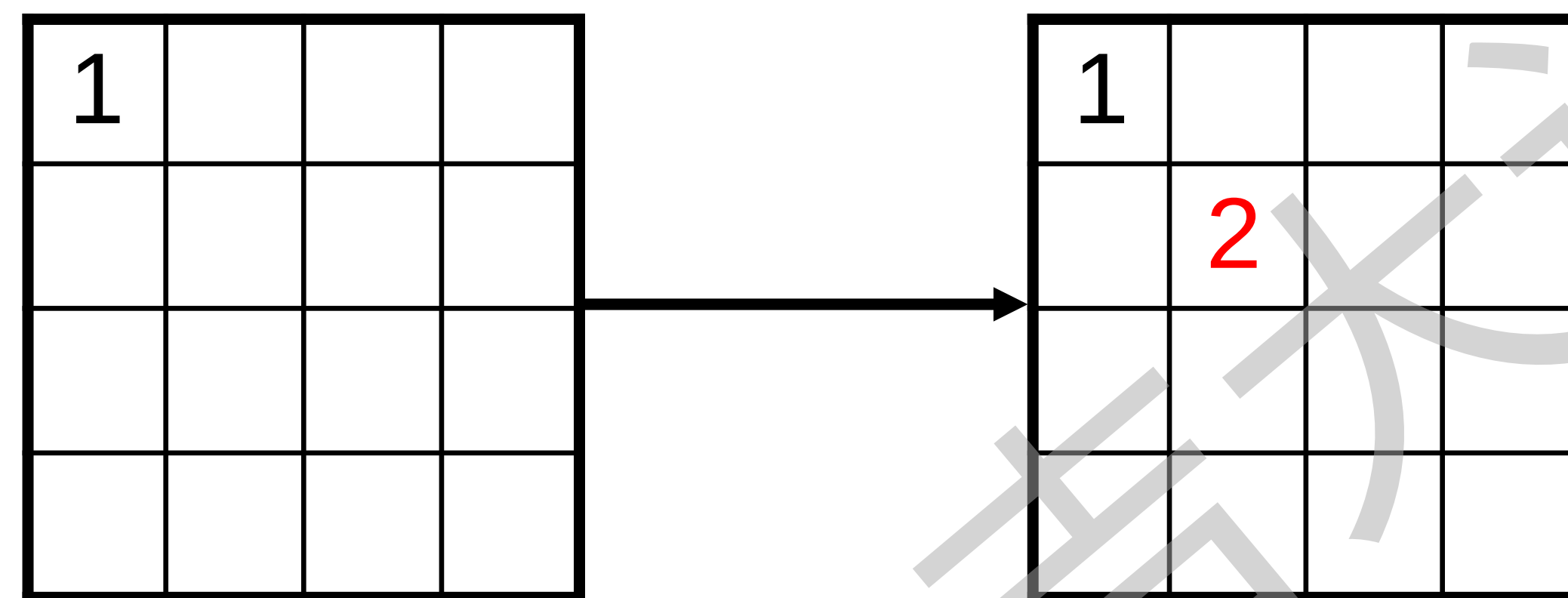


- $n=4$ 时, 叶结点个数= $4! = 24$, 解空间是从根结点到叶结点的所有路径。

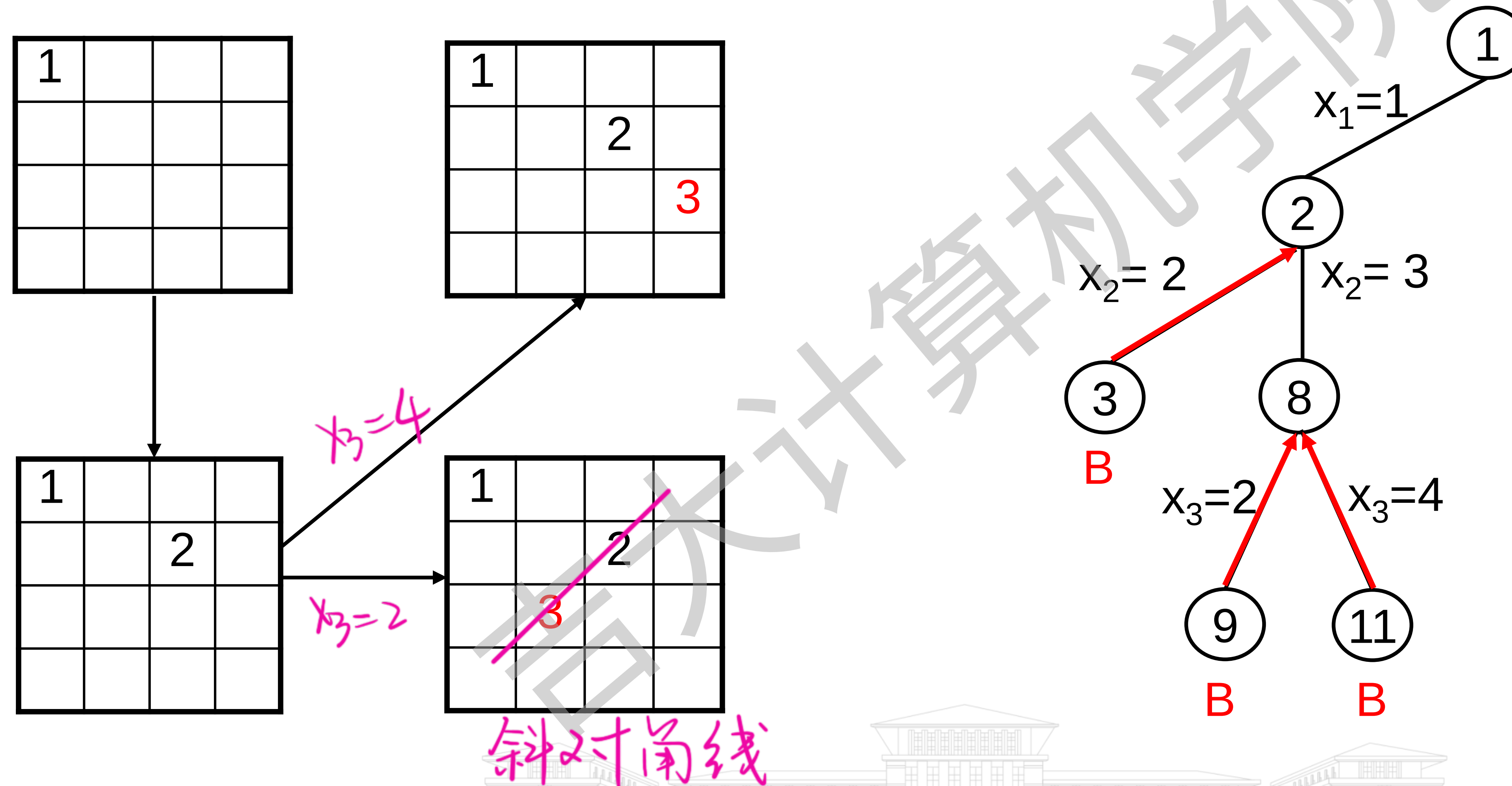


问题分析

- 开始把根结点作为唯一的活结点，根结点就成为E-结点而且路径为(**1**)；接着生成儿子结点，那么结点2被生成，这条路径为(**1, 2**)，即把皇后1放在第1列上。
- 结点2变成E-结点，它再生成结点3，路径变为(**1, 2, 3**)，结点3被杀死，此时回溯。

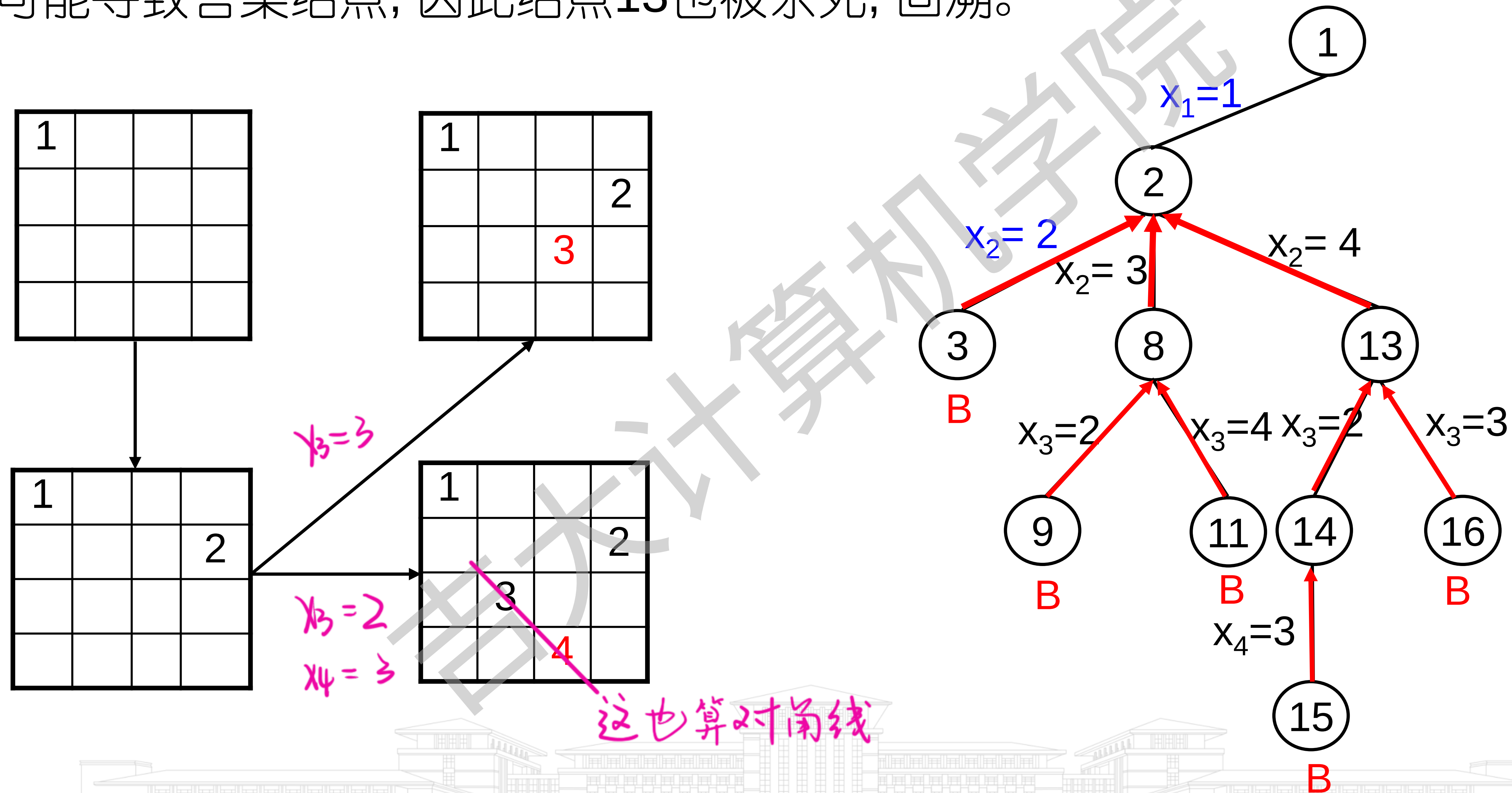


- 回溯到结点2生成结点8, 路径变为(1, 3), 则结点8成为E-结点, 它生成结点9和结点11都会被杀死, 所以结点8也被杀死, 应回溯。

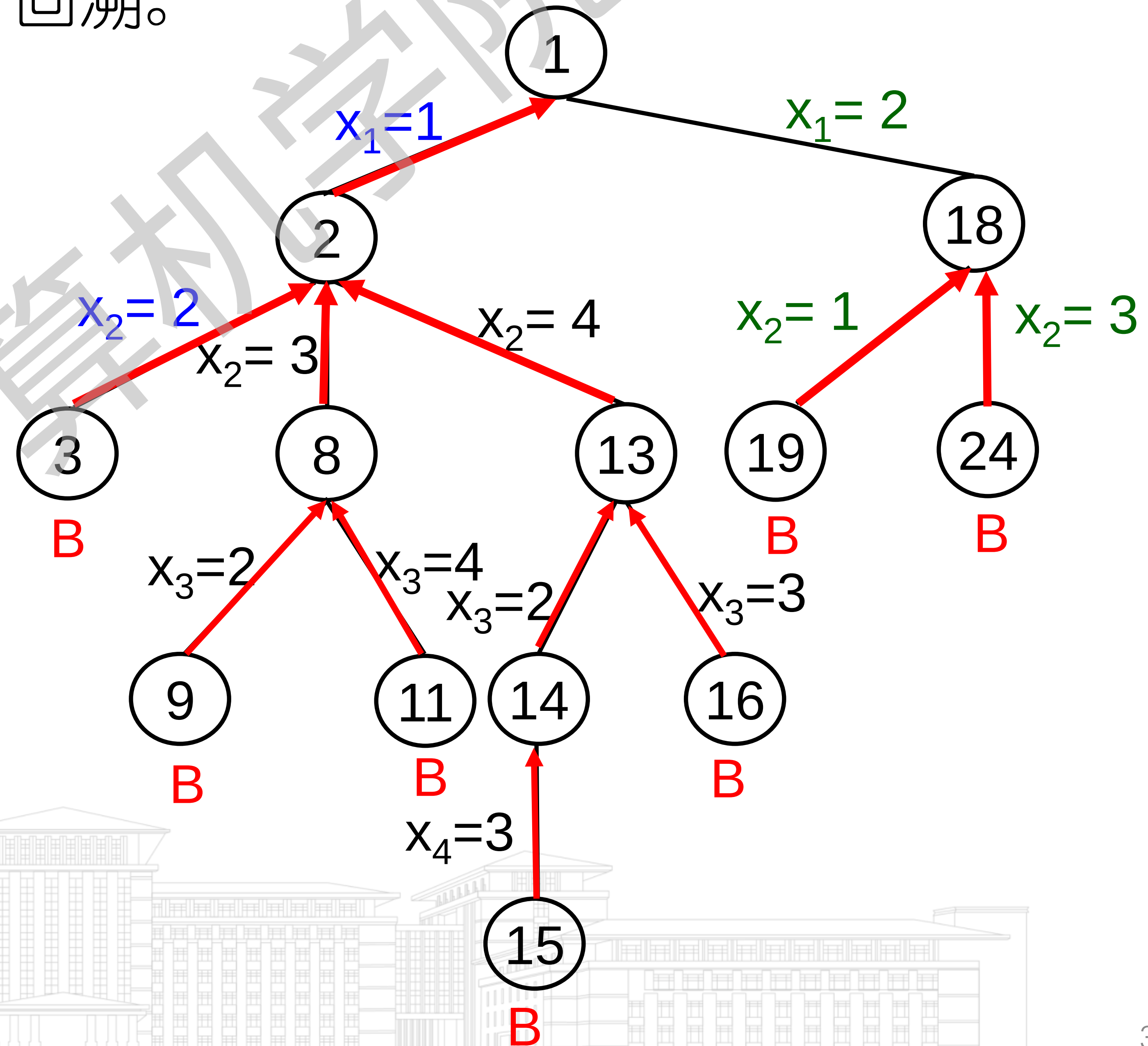
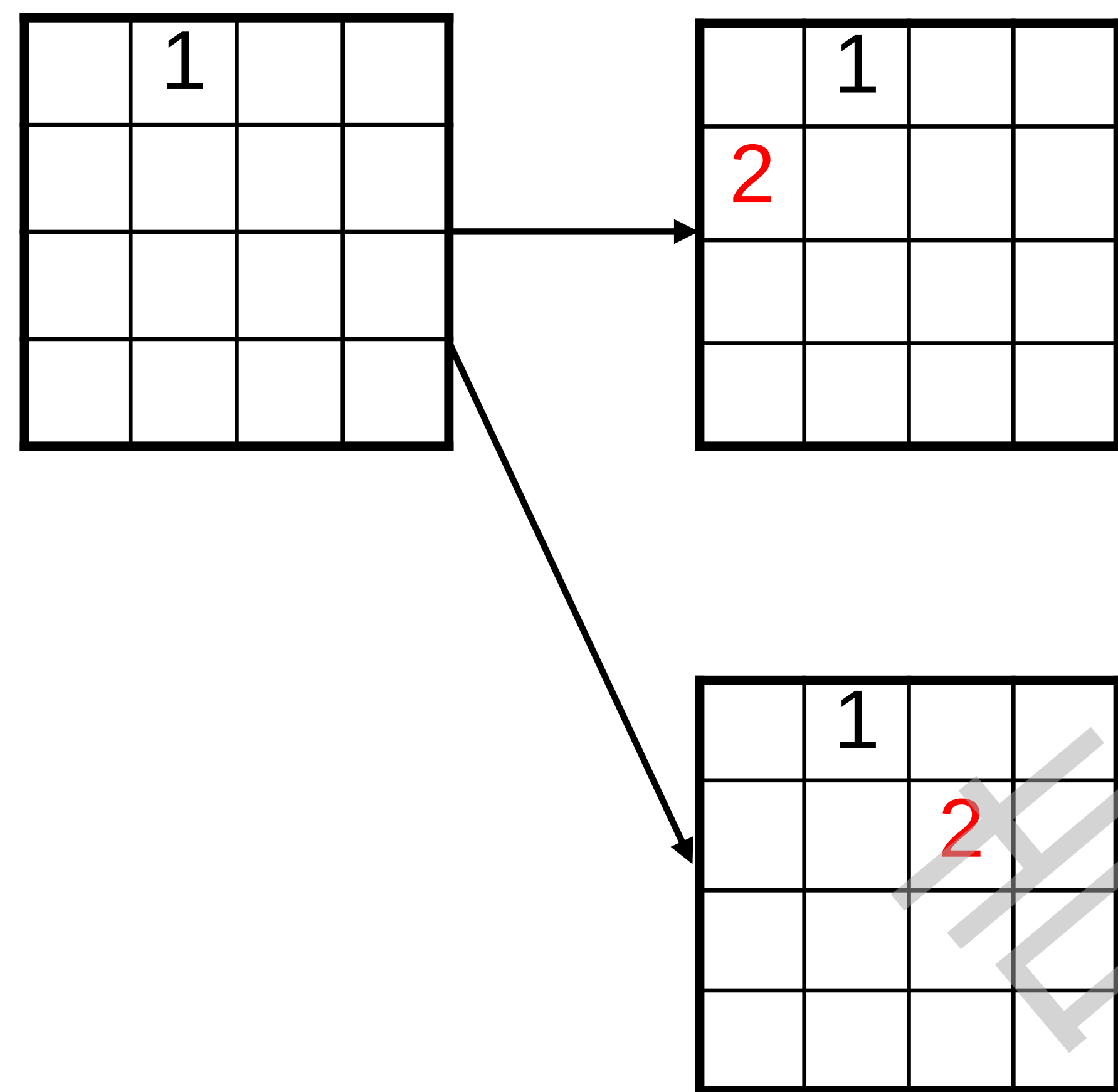




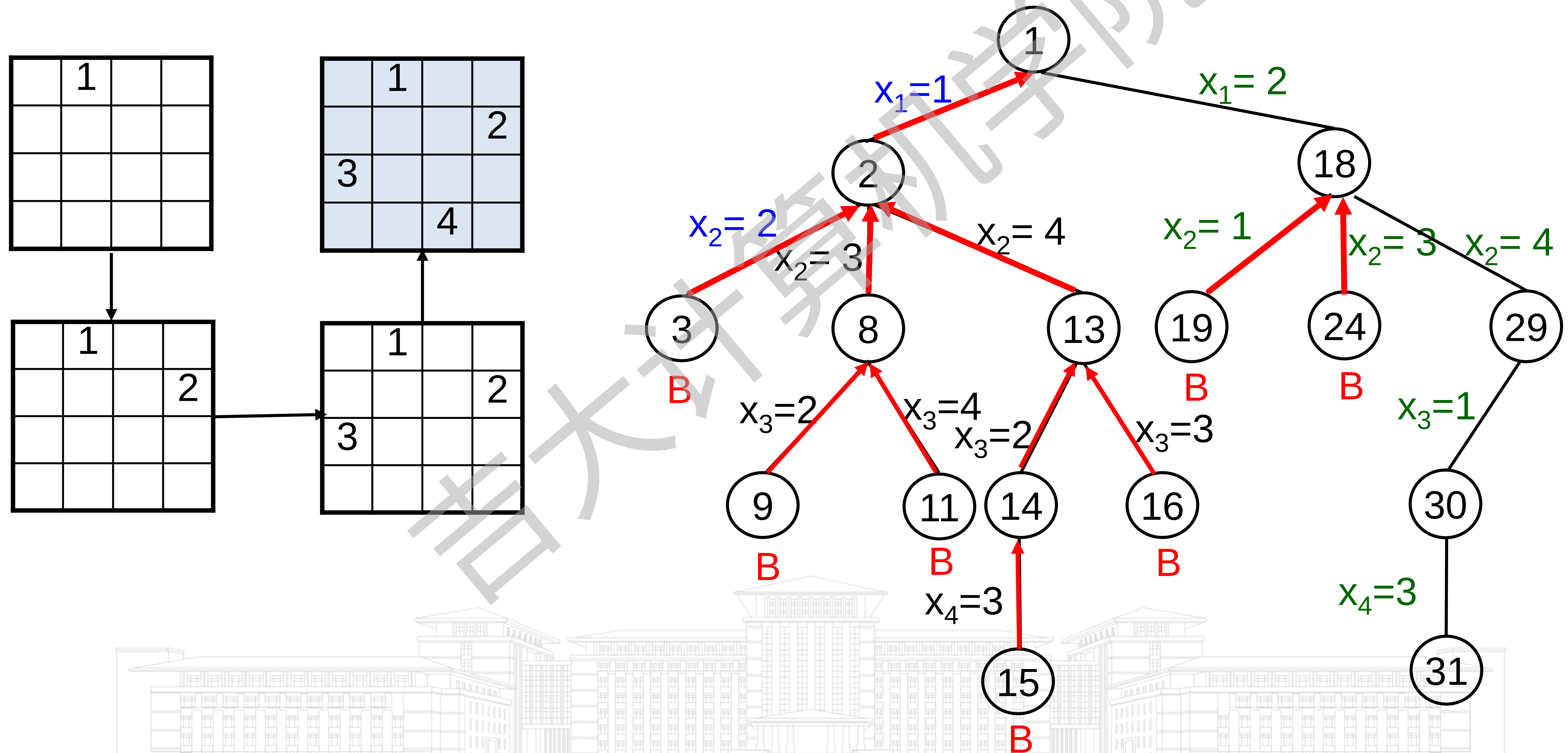
- 回溯到结点2生成结点13, 路径变为(1, 4), 结点13成为E-结点, 它的儿子不可能导致答案结点, 因此结点13也被杀死, 回溯。



- 结点2的所有儿子都不能导致答案棋盘格局, 因此结点2也被杀死; 再回溯到结点1生成结点18, 路径变为(2)。
- 结点18的儿子结点19、结点24被杀死, 回溯。



- 结点18生成结点29, 结点29成为E-结点, 路径变为(2,4)。
- 结点29生成结点30, 路径变为(2,4,1)。
- 结点30生成结点31, 路径变为(2,4,1,3), 找到一个4-皇后问题的可行解。



约束函数

- 在 n -皇后问题中, $(x_1, x_2, \dots, x_n)$ 表示一个解, x_i 表示第 i 个皇后放在第 i 行的列数。

- 同一条斜角线上的每个元素

- 由左到右具有相同的“行-列”值;
- 由右到左具有相同的“行+列”值。

a_{11}	a_{12}	a_{13}	a_{14}	a_{15}	a_{16}	a_{17}	a_{18}
a_{21}	a_{22}	a_{23}	a_{24}	a_{25}	a_{26}	a_{27}	a_{28}
a_{31}	a_{32}	a_{33}	a_{34}	a_{35}	a_{36}	a_{37}	a_{38}
a_{41}	a_{42}	a_{43}	a_{44}	a_{45}	a_{46}	a_{47}	a_{48}
a_{51}	a_{52}	a_{53}	a_{54}	a_{55}	a_{56}	a_{57}	a_{58}
a_{61}	a_{62}	a_{63}	a_{64}	a_{65}	a_{66}	a_{67}	a_{68}
a_{71}	a_{72}	a_{73}	a_{74}	a_{75}	a_{76}	a_{77}	a_{78}
a_{81}	a_{82}	a_{83}	a_{84}	a_{85}	a_{86}	a_{87}	a_{88}

从右到左

从左到右

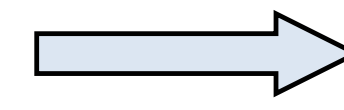
约束函数

- 设有两个皇后位于 $(i, X(i))$ 和 $(k, X(k))$ 位置上

$$i - X(i) = k - X(k)$$

$$i + X(i) = k + X(k)$$

位置



$$|X(i) - X(k)| = |i - k|$$

// 前 $k-1$ 行的皇后已经放置，现在确定第 k 行皇后欲放在 $X(k)$ 列上，是否可以？

- 约束函数PLACE(k)的设计思想：
 - 令 $X(k)$ 与 $X(i)$ 逐个比较， $i=1..k-1$ 。
 - 若存在 $X(k)=X(i)$ 或者 $|X(i)-X(k)|=|i-k|$ ，则返回false；否则返回true。



算法7.4 能否放置一个新皇后？

procedure PLACE(k)

//若一个皇后能放在第k行和第X(k)列, 则返回true, 否则返回false。

//X是全局数组, 进入此过程时已置入了前k个值, ABS是绝对值函数。

int i, k

i ← 1

while (i < k) do

if (X(i)=X(k) or $ABS(X(i)-X(k))=ABS(i-k)$)

then return false

endif

i ← i+1

repeat

return true

end PLACE

算法7.5 n-皇后问题的回溯算法

记住此算法的结构

```

procedure NQUEENS(n)
int k, n, X(1:n)
X(1) ← 0 ; k ← 1
while (k>0) do
    X(k) ← X(k)+1
    while ( X(k) ≤ n and not PLACE(k) ) do
        X(k) ← X(k)+1; repeat //当前列X(k)不能放皇后k时，放到下一列
    if(X(k) ≤ n)
    then if(k=n)
        then print (X)
        else k ← k+1; X(k) ← 0 //准备求解下一个皇后
        endif
    else k ← k-1; //没有合适的位置，回溯
    endif
repeat
end NQUEENS
    
```



```
#define N 8 // 假设棋盘大小为8x8，可以根据需要调整

int *X; // 全局数组X存储每行皇后的列位置

bool PLACE(int k) {
    for (int i = 0; i < k; i++) {
        if ((X[i] == X[k]) || abs(X[i] - X[k]) == abs(i - k)) {
            return false;
        }
    }
    return true;
}

void NQUEENS(int n) {
    X = (int *)malloc(n * sizeof(int)); // 动态分配内存
    int k = 0;
    X[0] = 0; // 初始化第一个皇后的位置

    while (k >= 0) {
        X[k] += 1; // 尝试下一个列位置
        while (X[k] < n && !PLACE(k)) { // 当前列不满足条件时尝试下一列
            X[k] += 1;
        }
        if (X[k] < n) {
            if (k == n - 1) { // 找到一个完整的解
                for (int i = 0; i < n; i++) {
                    printf("%d ", X[i]);
                }
                printf("\n");
            } else {
                k++;
                X[k] = -1; // 初始化下一行的皇后位置为非法值
            }
        } else {
            k--; // 回溯
        }
    }
    free(X); // 释放动态分配的内存
}
```

全局变量 `x`：用于存储每一行皇后所在的列位置。

函数 `PLACE`：检查在第 `k` 行放置皇后是否合法。如果与之前的任意皇后在同一列或同一对角线上，则返回 `false`；否则返回 `true`。

函数 `NQUEENS`：主函数，使用回溯法求解n-皇后问题。

- 使用 `while` 循环和递增列索引来尝试每个可能的皇后位置。
- 如果找到合法位置并且当前是最后一行，则打印解决方案。
- 如果找到合法位置但不是最后一行，则继续处理下一行。
- 如果没有找到合法位置，则回溯至上一行。

内存管理：通过 `malloc` 动态分配数组 `x` 的内存，并在函数结束时通过 `free` 释放。

4-皇后问题实例

- 当前 $k=3$, $X(3)=0$

i	1	2	3	4
$X(i)$	1	3	5	

1			

1			
		2	2
3	3	3	3

- 回溯到 $k=2$, 修改 $X(2)$

i	1	2	3	4
$X(i)$	1	4	0	

- 令 $k=3$, 重新开始构造 $X(3)$

1→2→3→4都不行



8-皇后问题的效率估计

- 在8-皇后问题中, $(x_1, x_2, \dots, x_8)$ 表示一个解, x_i 表示第 i 个皇后放在第 i 行的列数。
 - 显式: $S_i = \{1, 2, 3, 4, 5, 6, 7, 8\}$, $1 \leq i \leq 8$, 且没有两个 x_i 可以相同
 - 隐式: 没有两个皇后可以在同一条斜对角线上
- 状态空间树结点个数: $1 + 8 + 8 * 7 + 8 * 7 * 6 + \dots + 8 * 7 * 6 * 5 * 4 * 3 * 2 * 1 = 69281$
- 约束函数 **PLACE** 实现隐式约束条件, 会生成多少个结点?

使用蒙特卡罗方法估计



8-皇后问题的不受限结点的估计值

只能放在空位位置

	1						
			2				
3							
		4					
			5				

8 1+
5 8+
4 8*5+
3 8*5*4+
2 8*5*4*3+
8*5*4*3*2
=1649

1							
						2	
				3			
		4					
					5		
	6						
			7				

8
6
4
2
1
1
1
=1401

多次实验后取平均值1625

不受限结点的估计数大约是8-皇后状态空间树的结点总数的

$$1625/69281=2.34\%$$





7.4 子集和问题

- 问题描述
- 约束函数
- 算法7.6 子集和的回溯递归算法
- 实例分析
- 效率估计



问题描述

- 子集和问题：
 - 假定有 n 个不同的正数 $W(1:n)$ ，找出这些数中所有使得和为 M 的组合。
元素 $W(i)$ 称为权。
- 回溯法求解：
 - n -元组 $(x_1, \dots, x_n)$
 - 显示约束条件： $x_i \in \{0, 1\}, 1 \leq i \leq n$ ；如果选择 w_i ，则 $x_i = 1$ ；否则 $x_i = 0$
 - 隐式约束： $\sum w_i x_i = M, 1 \leq i \leq n$





约束函数

设一维数组X和W，X表示n-元组解向量，W记录n个正数

- 当满足 $\sum_{i=1}^k W(i) X(i) + \sum_{i=k+1}^n W(i) \geq M$
 - $X(1), \dots, X(k)$ 能导致一个答案结点
- 设W(i)按非降次序排列，那么当 $\sum_{i=1}^k W(i) X(i) + W(k+1) > M$
 - $X(1), \dots, X(k)$ 不能导致一个答案结点。

- 综上，约束函数 $B_k(X(1), \dots, X(k)) = \text{true}$ ，当且仅当：

W(i)按非降次序排列

$$\sum_{i=1}^k W(i) X(i) + \sum_{i=k+1}^n W(i) \geq M \quad \text{且} \quad \sum_{i=1}^k W(i) X(i) + W(k+1) \leq M$$

s

r

约束函数剪枝

左子树约束(选择当前元素 $w(k)$):

$$S + w(k) \leq M \quad (\text{当前总和不超过 } M)$$

$$S + r \geq M \quad (\text{剩余元素足够达到 } M, \text{ 其中 } r \text{ 是剩余元素的总和})$$

右子树约束(不选择当前元素 $w(k)$):

$$S + r - w(k) \geq M \quad (\text{不选 } w(k) \text{ 后剩余元素仍足够})$$

$$S + w(k+1) \leq M \quad (\text{选择下一个元素不会超过 } M)$$

算法7.6 子集和的回溯递归算法

Procedure SUMOFSUB(s,k,r)

//找出 $W(1:n)$ 中和为 M 的所有子集。 $s = \sum_{j=1}^{k-1} W(j) X(j)$ 且 $r = \sum_{j=k}^n W(j)$

//进入此过程时 $X(1), \dots, X(k-1)$ 的值已确定。

//这些 $W(j)$ 按非降次序排列。 $W(1) \leq M, \sum_{i=1}^n W(i) \geq M$

integer $W(1:n), M, n;$

boolean $X(1:n)$

integer s, k, r



//生成左儿子。由于 $B_{k-1}=\text{true}$ ，因此 $s+W(k)\leq M$ 且 $s+r \geq M$

$X(k)\leftarrow 1$

if $s+W(k) = M$

then print(X)

else if $s+W(k)+W(k+1) \leq M$ then call SUMOFSUB($s+W(k)$, $k+1$, $r-W(k)$)

endif

endif

//生成右儿子和计算 B_k 的值

If $s+r-W(k) \geq M$ and $s+W(k+1) \leq M$

then $X(k)\leftarrow 0$; call SUMOFSUB(s , $k+1$, $r-W(k)$)

endif

end SUMOFSUB

左子树-递归入口

右子树-递归入口

初始调用: $\text{SUMOFSUB}(0, 1, \sum_{1 \leq i \leq n} W(i))$

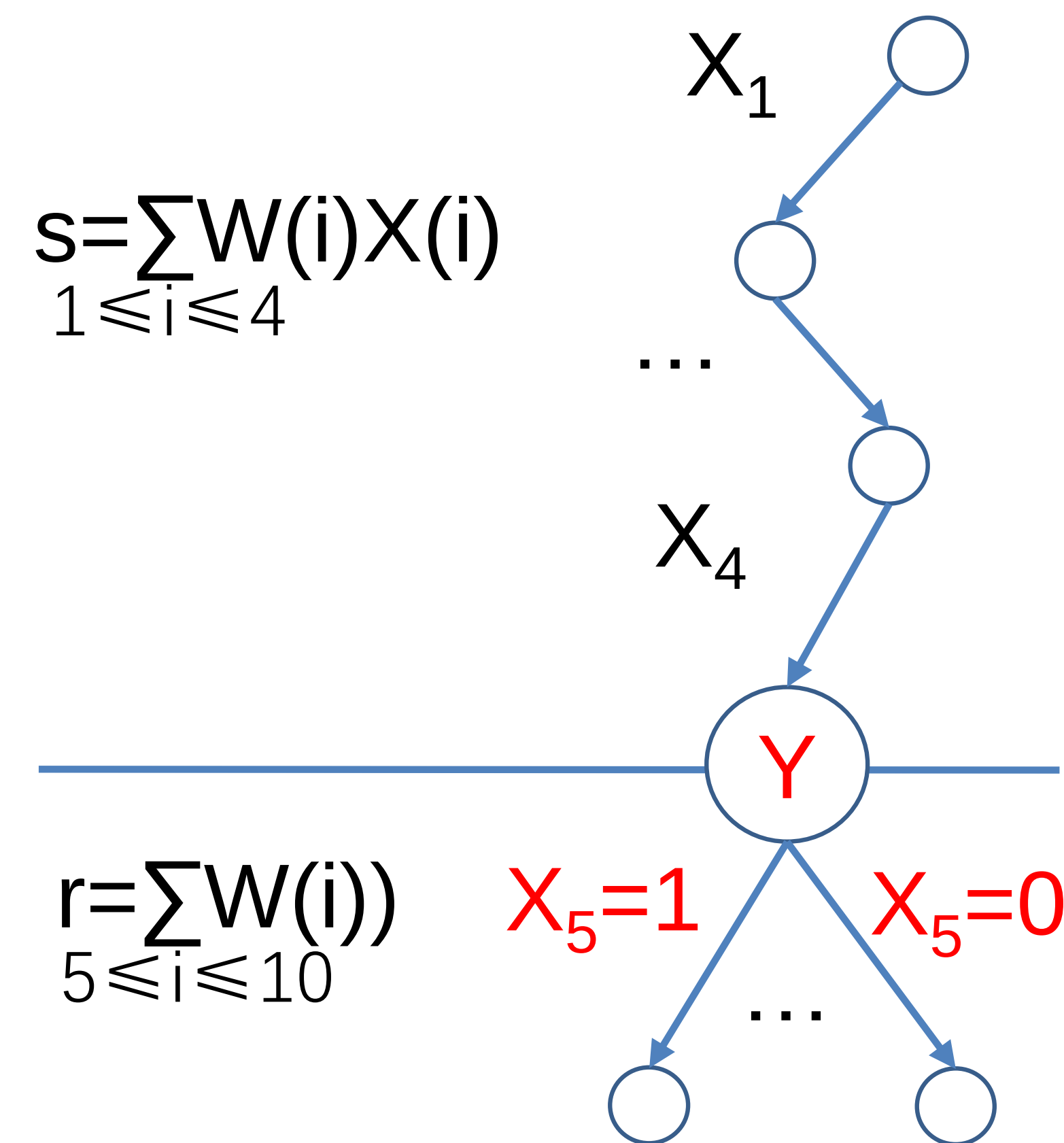
思考1: 如果不将 W 预排序, 算法怎样设计? 算法效率怎样变化?

思考2: 为什么左子树简化了约束条件的判定?



设当前 $n=10$

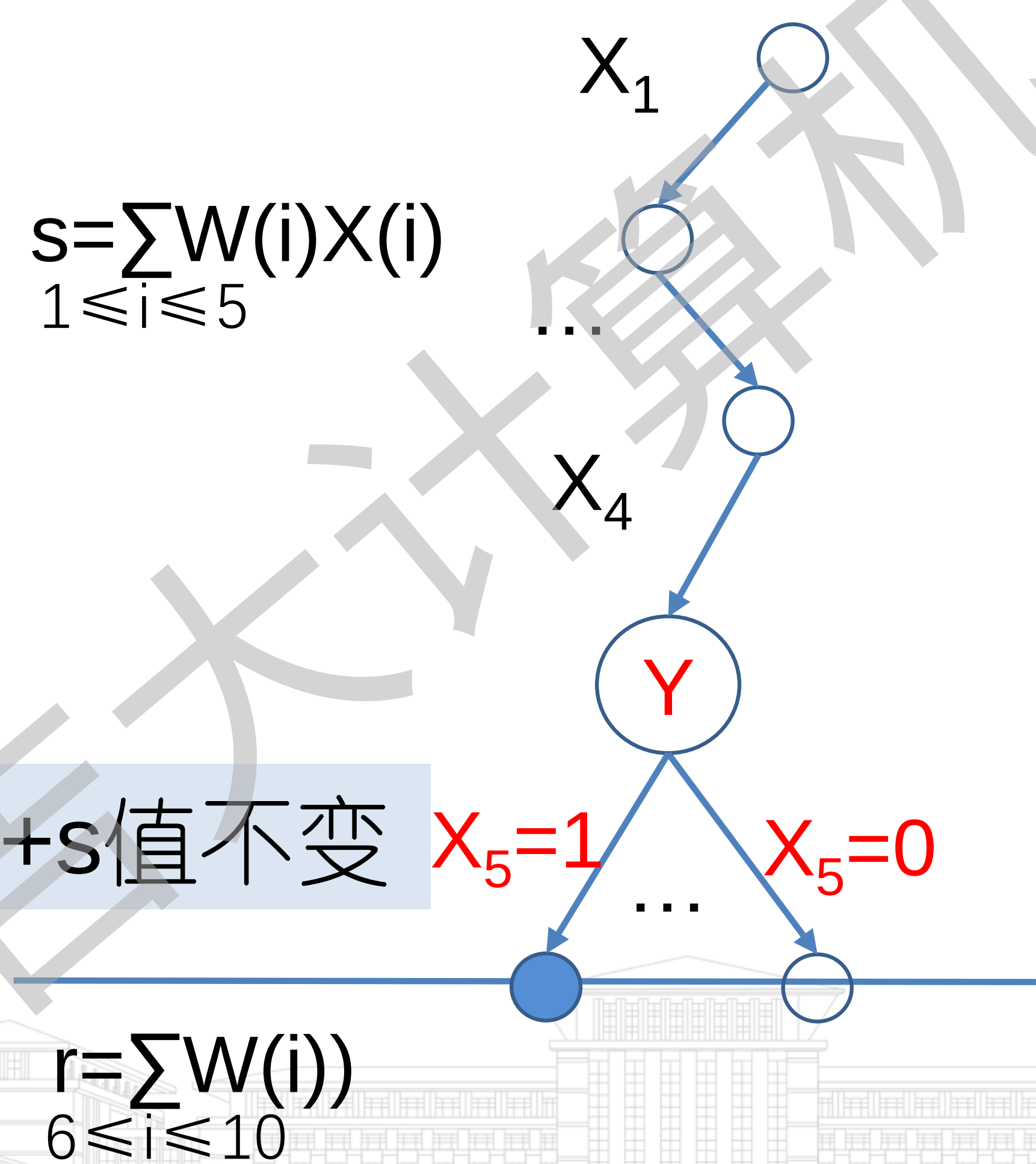
SUMOFSUB(s, 5, r)



访问Y的左儿子结点

如果 $s + W(5) + W(6) \leq M$,

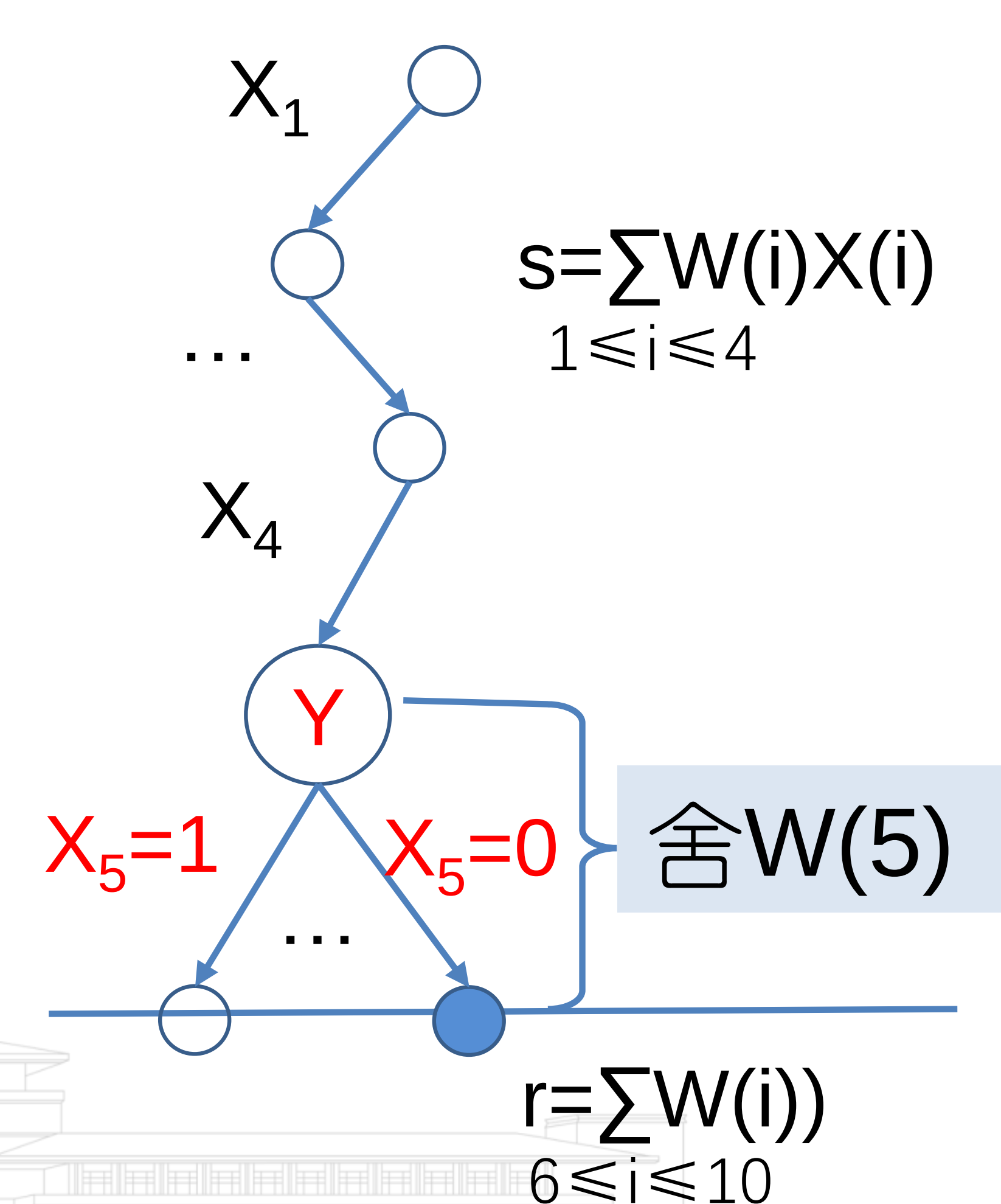
调用SUMOFSUB($s+W(5)$, 6, $r-W(5)$)



访问Y的右儿子结点

如果 $s+r-W(5) \geq M$ 且 $s+W(6) \leq M$,

调用SUMOFSUB(s, 6, r-W(5))




```
// 假设w和n是全局变量或者作为参数传递给函数
extern int w[], n, M;

bool SUMOFSUB(int s, int k, int r, int *x) {
    // 生成左儿子结点
    x[k] = 1;
    if (s + w[k] == M) {
        // 打印解向量x, 这里简化处理, 可以根据需要修改打印逻辑
        for (int i = 0; i <= k; i++) {
            printf("%d ", x[i]);
        }
        printf("\n");
    } else if (s + w[k] + w[k+1] <= M) {
        if (!SUMOFSUB(s + w[k], k + 1, r - w[k], x)) {
            return false;
        }
    }

    // 生成右儿子结点
    if ((s + r - w[k] >= M) && (s + w[k+1] <= M)) {
        x[k] = 0;
        if (!SUMOFSUB(s, k + 1, r - w[k], x)) {
            return false;
        }
    }

    return true;
}

// 示例调用:
// 需要先对w数组进行非降序排序, 并计算r (所有剩余元素的和)
// int x[100]; // 假设n不会超过100, 或根据实际情况调整数组大小
// int r = 0;
// for (int i = 0; i < n; i++) {
//     r += w[i];
// }
// SUMOFSUB(0, 0, r, x);
```

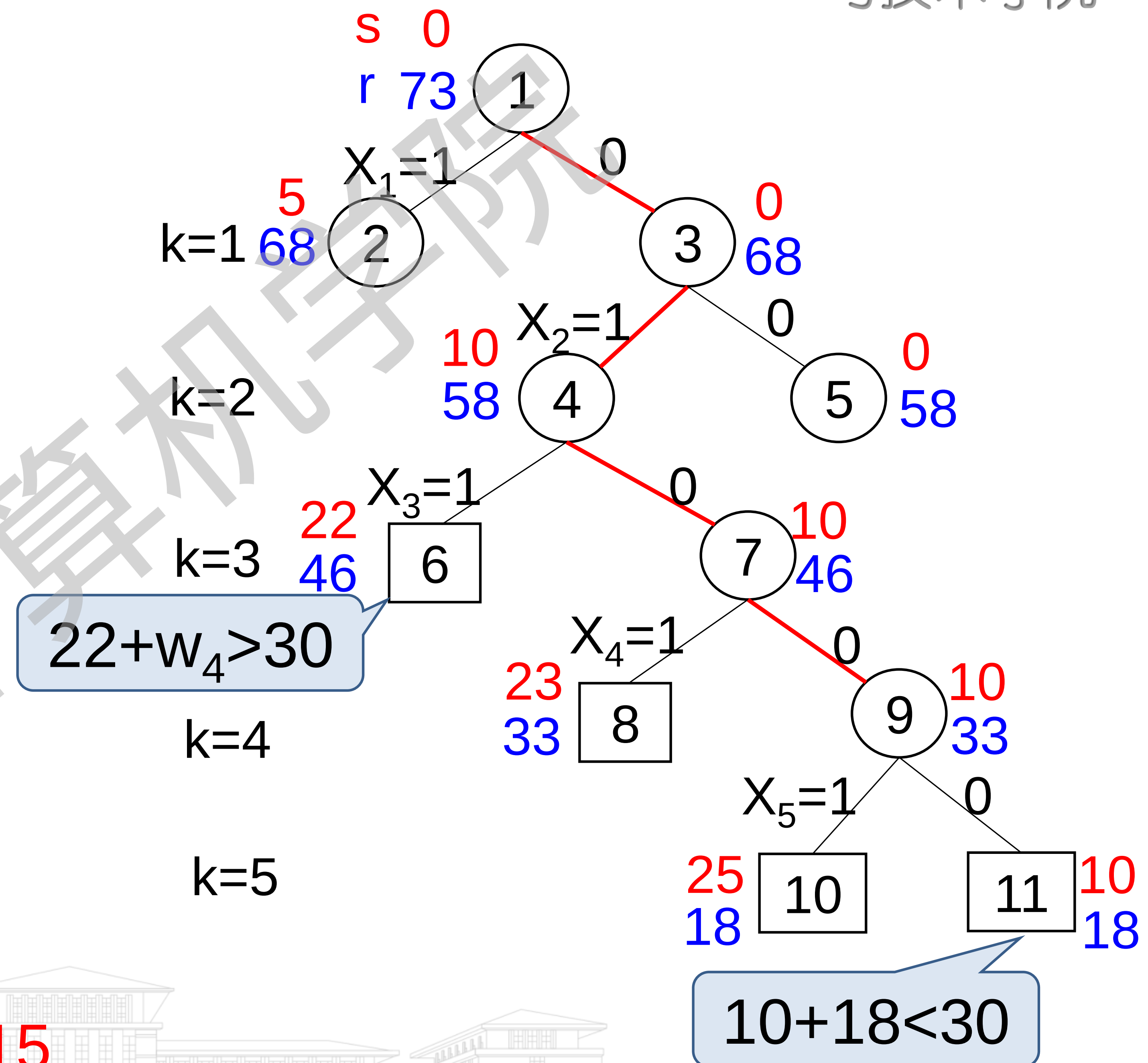


子集和问题效率估计

	W_1	W_2	W_3	W_4	W_5	W_6
$M=30$	5	10	12	13	15	18

$$B_k: s+W(k+1) \leq M \text{ 且 } s+r \geq M$$

- 方形结点表示不满足约束条件
- 随机选择路径为：1-3-4-7-9
- $n=6$ 的解空间树共计 $64+63$ 个结点
- 不受限界结点的估计数：
 - $m=1+2+2*2+2*2*1+2*2*1*1+0=15$



实例运行结果

- $n=6, M=30, W=(5,10,12,13,15,18)$
- 使用约束函数前，状态空间树中所有结点都会被访问到，叶结点(解状态)个数为 $2^6=64$ 个，部分解结点63个。
- 使用约束函数后，动态树一共生成26个结点，寻找/提前寻找到3个答案结点

和估计结果接近





7.5 图着色问题

- 问题描述
- 图的 m -着色判定问题
- 解空间树
- 算法7.7 约束函数
- 算法7.8 图着色判定问题的回溯算法
- 实例分析





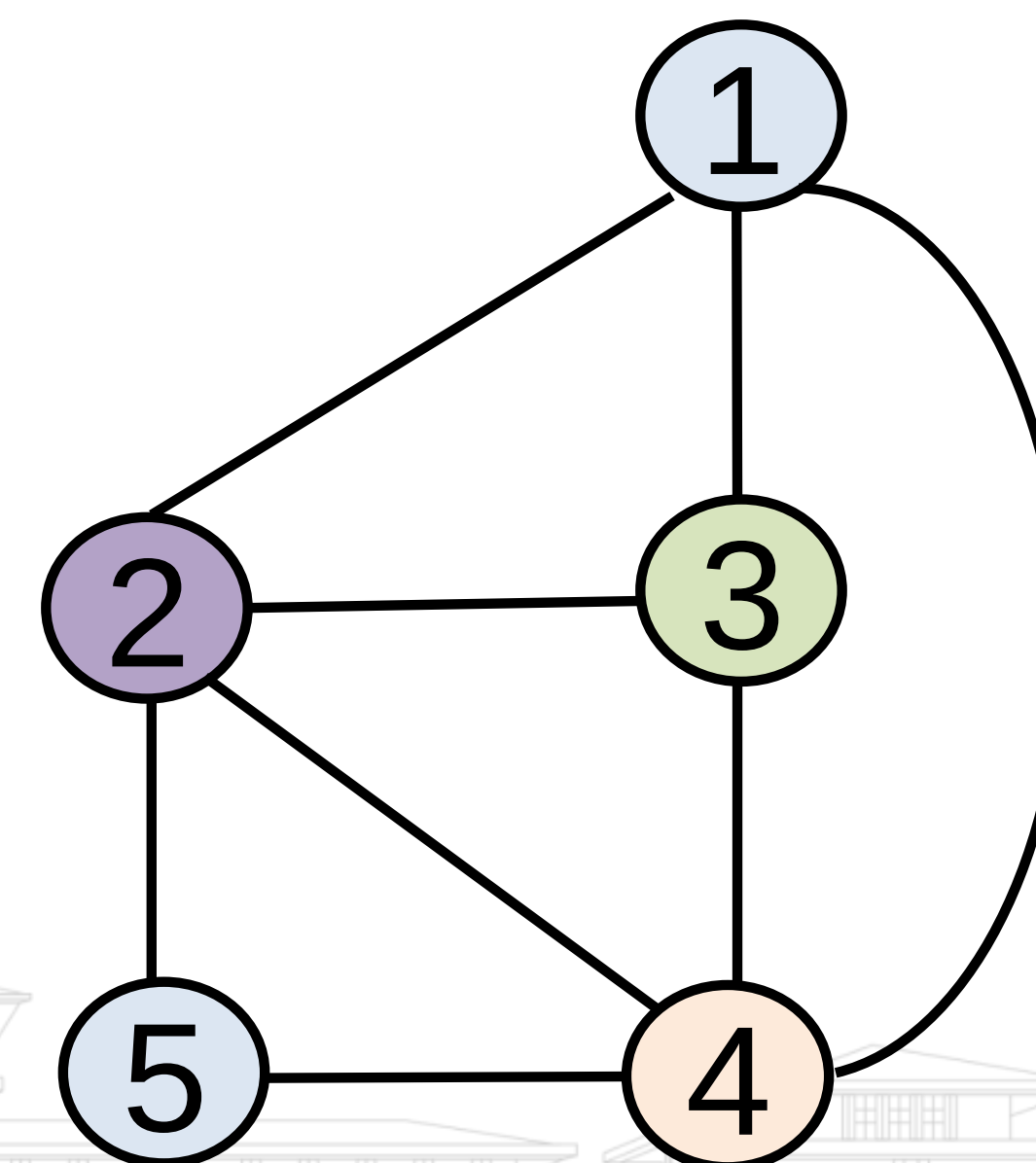
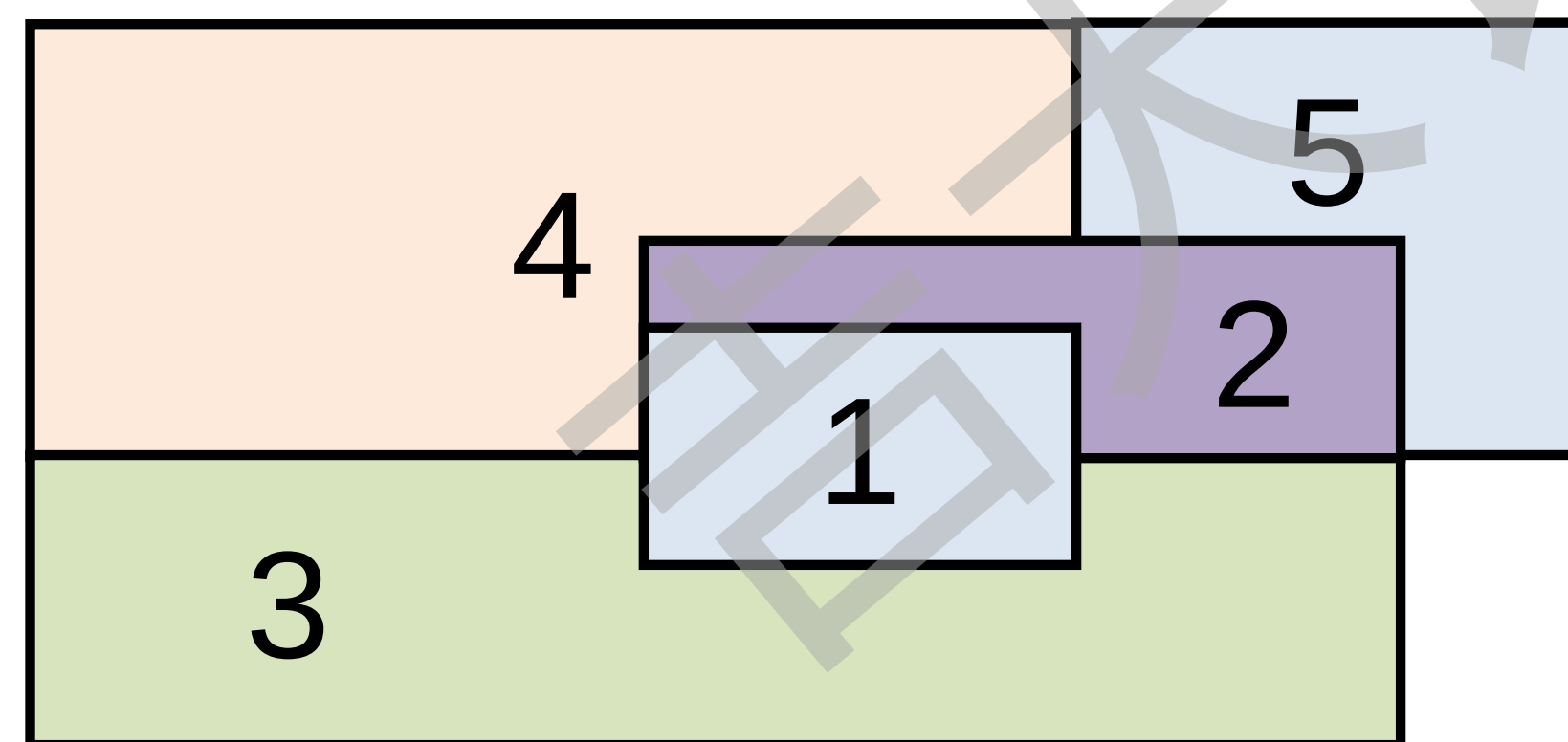
问题描述

- 图着色问题(Graph Coloring Problem, GCP) 又称着色问题, 是最著名的NP-完全问题之一。
- 数学定义: 给定无向连通图 $G=(V,E)$, 其中 V 为顶点集合, E 为边集合, 用不同的颜色给图中顶点着色, 要求任何两个相邻顶点的着色不同。
- 问: 最少需要多少种颜色?



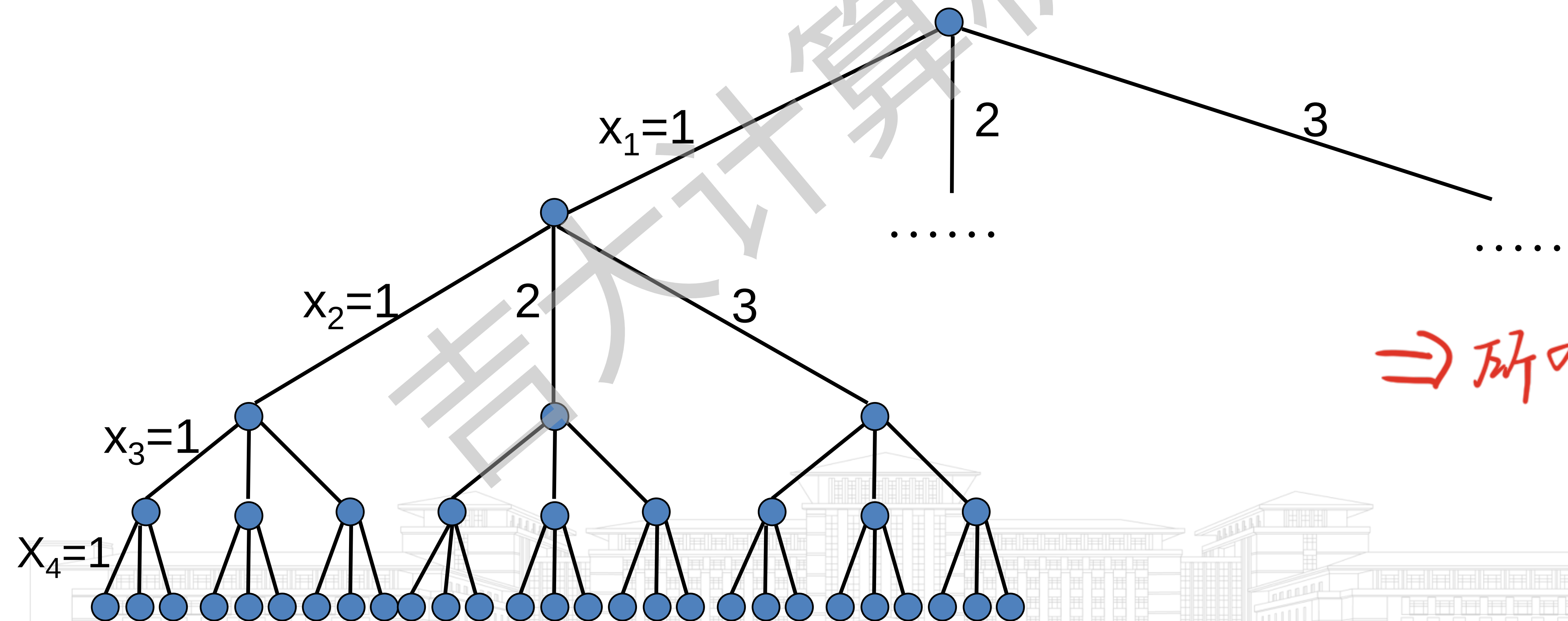
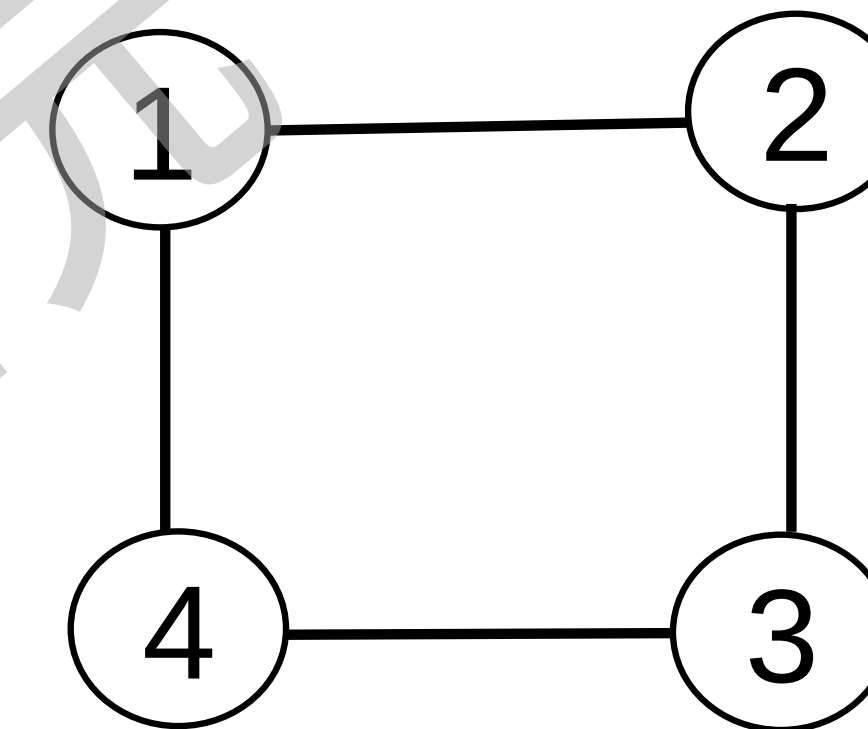
图的m-着色判定问题

- 给定无向连通图 $G=(V,E)$ ，其中 V 为顶点集合， E 为边集合，用 m 种不同颜色给图中顶点着色，问：是否存在任何两个相邻顶点颜色不同的着色方案？
- 本节用回溯来解决图的 m 着色判定问题，如果判定答案为“是”，要求给出着色方案。



解空间树

- 考虑 $n=4$ (4个顶点)的连通图, $m=3$ (3种颜色)
 - n 元组表示: $X = (x_1, \dots, x_4)$, x_i 表示结点 i 的颜色。
 - 显式: $1 \leq x_i \leq 3$ 。
 - 隐式: 若结点 i 和 j 之间有边存在, 则 $x_i \neq x_j$ 。



⇒ 所有的可能性



算法7.7 判断顶点k的着色是否合法

procedure OK(k)

// C是全局数组, 进入此过程时已置入了前k个值

int i, k

$i \leftarrow 1$

while ($i < k$) do

if (i和k之间有边存在 and $C(i) = C(k)$)

then return false

endif

$i \leftarrow i + 1$

repeat

return true

end OK

约束函数

加入隐式约束条件。


```
// 假设图用邻接矩阵表示，graph[i][j] == 1 表示顶点 i 和 j 之间有边相连
extern int graph[100][100]; // 假设最多100个顶点，或根据实际情况调整大小
extern int n; // 顶点数量

bool OK(int k, int *colors) { // colors数组记录每个顶点的颜色
    for (int i = 0; i < k; i++) {
        if (graph[i][k] == 1 && colors[i] == colors[k]) { // 如果顶点i和k之间有边且颜色相同
            return false;
        }
    }
    return true;
}
```

`graph` 是一个二维数组，用来表示图的邻接矩阵。如果 `graph[i][j] == 1`，则表示顶点 `i` 和顶点 `j` 之间存在一条边。

`n` 是图中顶点的数量。

`colors` 是一个一维数组，其中 `colors[i]` 表示顶点 `i` 的颜色编号。

函数 `OK` 接受两个参数：`k` 表示当前正在考虑为其着色的顶点索引，`colors` 是存储各个顶点颜色的数组。

函数遍历所有在 `k` 之前已经着色的顶点（从 `0` 到 `k-1`），检查是否存在与顶点 `k` 相邻且颜色相同的顶点。如果找到这样的顶点，则返回 `false`；否则，在循环结束后返回 `true`。

算法7.8 图着色判定问题的回溯算法



```
Procedure M Coloring(V,E,C,n,m)
// 图G=(V,E),n个顶点, m种颜色
C(1:n) ← 0; k ← 1 //C记录决策序列, 从第一个顶点开始
while (k ≥ 1)
    C(k) ← C(k) + 1
    while (not OK(k) and C(k) ≤ m) do C(k) ← C(k) + 1 repeat
    if C(k) ≤ m then
        if k = n then print(C); return true //全部着色,打印
        else k ← k + 1; C(k) ← 0 //准备为下一个顶点着色
    endif
    else k ← k - 1 //顶点k无法着色, 回溯
endif
repeat //k=0表示整颗树遍历完毕
return false
END M Coloring
```



```
// 假设图G由邻接矩阵a表示，n为顶点数，m为颜色数
extern int n, m;
extern bool a[50][50]; // 假设最多50个顶点

bool OK(int k, int *C) {
    for (int i = 0; i < k; i++) {
        if (a[i][k] && C[i] == C[k]) { // 如果顶点i和k之间有边且颜色相同
            return false;
        }
    }
    return true;
}

void M Coloring(int m, int n, int *C) {
    int k = 0;
    C[k] = 0; // 初始化第一个顶点的颜色

    while (k >= 0) {
        C[k] = C[k] + 1; // 尝试下一个颜色

        while (C[k] <= m && !OK(k, C)) { // 找到一个合法的颜色
            C[k] = C[k] + 1;
        }

        if (C[k] <= m) {
            if (k == n - 1) { // 所有顶点都已着色
                for (int i = 0; i < n; i++) {
                    printf("Vertex %d has color %d\n", i, C[i]);
                }
                return; // 打印结果并结束
            } else {
                k = k + 1; // 移动到下一个顶点
                C[k] = 0; // 初始化下一个顶点的颜色
            }
        } else {
            k = k - 1; // 回溯
        }
    }
}
```

a 是图的邻接矩阵，用来表示图中各顶点之间的连接情况。如果顶点 i 和顶点 j 之间有边，则 a[i][j] = true。

n 是图的顶点数，m 是可用的颜色数。

c 数组用来存储每个顶点的颜色编号。

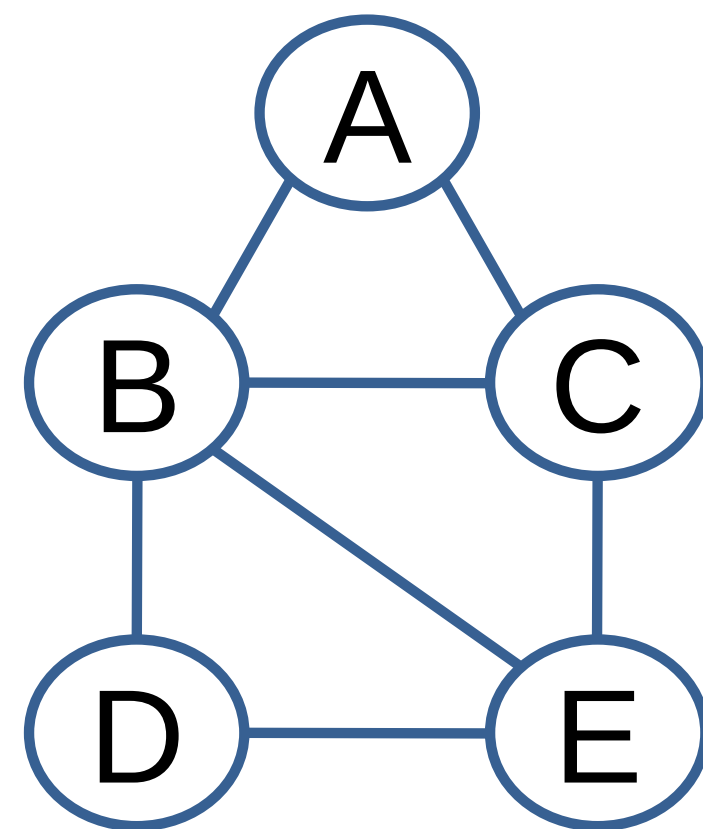
OK 函数检查当前顶点的颜色是否与之前已经着色的所有相邻顶点的颜色不同。

M Coloring 函数实现了回溯法来尝试给每个顶点着色，并在找到一种可行方案时打印出来。

实例分析



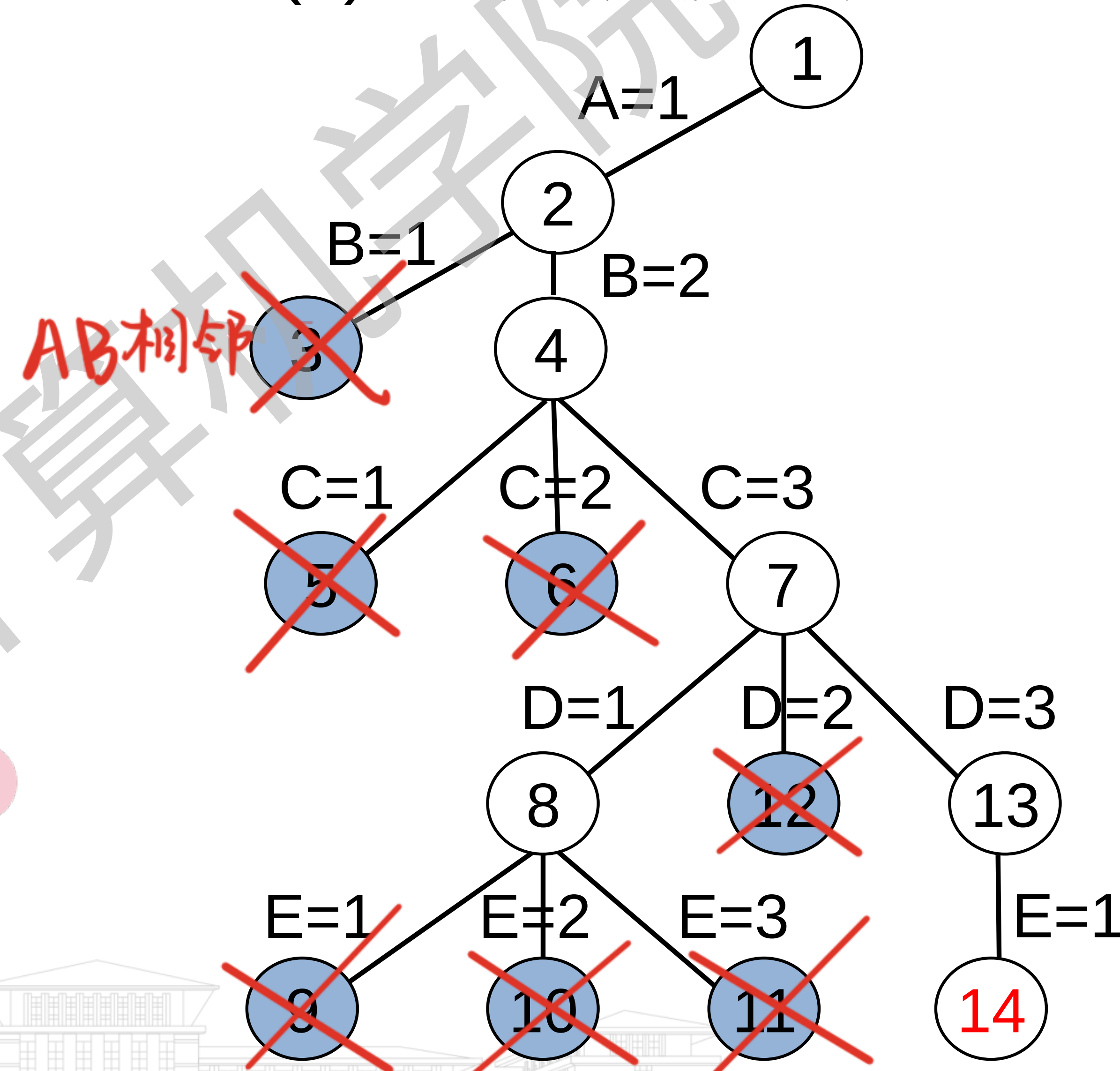
(a) 一个无向图



$n=5$ 个顶点的无向图, $m=3$, 对应的空间树是完全 m 叉树, 最后一层有多少个叶子结点?

$$m^n$$

(b) 回溯法搜索空间



思考1: 基于判定算法求最优解的时间复杂度是多少?

思考2: 图着色问题是否适用于蒙特卡洛方法估计?



7.6 小结

- 回溯法适用的问题
 - 多阶段决策问题/组合问题满足多米诺性质
- 回溯法的设计思想概述
 - 确定解向量： n -元组/ k -元组
 - 分解约束条件：显示&隐式
 - 确定解空间树
 - 设计约束函数**B**
 - 深度优先方式搜索树





7.6 小结

- 解空间树的分类
 - 集合树：问题的解是对已知集合元素的取舍
 - 如子集和问题，0/1背包问题
 - 排列树：问题的解是对已知集合元素的排列
 - 如n-皇后问题，图着色判定问题





7.6 小结

- 回溯法的效率问题
 - 解空间树的大小：决定最坏情况
 - 约束函数**B**的剪枝能力：决定动态树
 - 求问题的所有解时，可以用蒙特卡洛方法估计算法效率
- 回溯法的改进
 - 根据树的分支情况设计优先策略，如优先搜索结点少/解多的分支
 - 利用搜索树的对称性对子树进行剪裁
 - 分解成子问题，求解完子问题的解之后合并出原问题的解





7.6 小结

- 7.1 回溯一般方法
 - 掌握回溯法适用的问题特点、解空间树、状态结点、动态树结点生成方式等基本知识。能掌握回溯法求解问题的设计思想和一般方法。
- 7.2 回溯法的效率估计
 - 掌握影响回溯法效率的要素和蒙特卡洛方法的设计思想





7.6 小结

- 7.3 n-皇后问题
- 7.4 子集和问题
- 7.5 图着色问题
 - 掌握解空间树、约束函数和回溯算法设计，运用蒙特卡洛方法对算法效率进行分析
 - 掌握经典问题的回溯法求解方法，掌握约束函数优化思想，提高回溯算法剪枝能力，提高算法效率

能够识别出适合回溯法的可计算性问题、独立设计算法和分析算法复杂度。





本章结束

